



Hackathon 2024

Sketching with Electronics

Arduino: Open Source Imagination



Daniel Spikol DIKU HCC

Alex, Andrea, Casper, Jeppe, Lucas, Mathilde, Thea, & Vicktoria

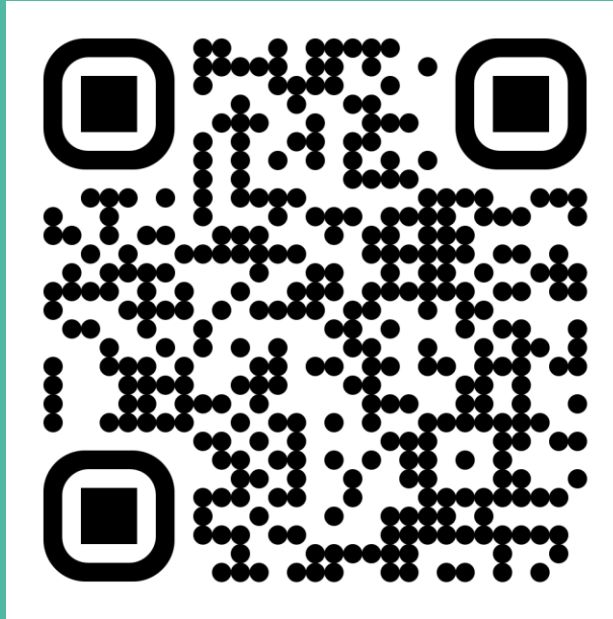


UNIVERSITY OF
COPENHAGEN



Join Absalon course:

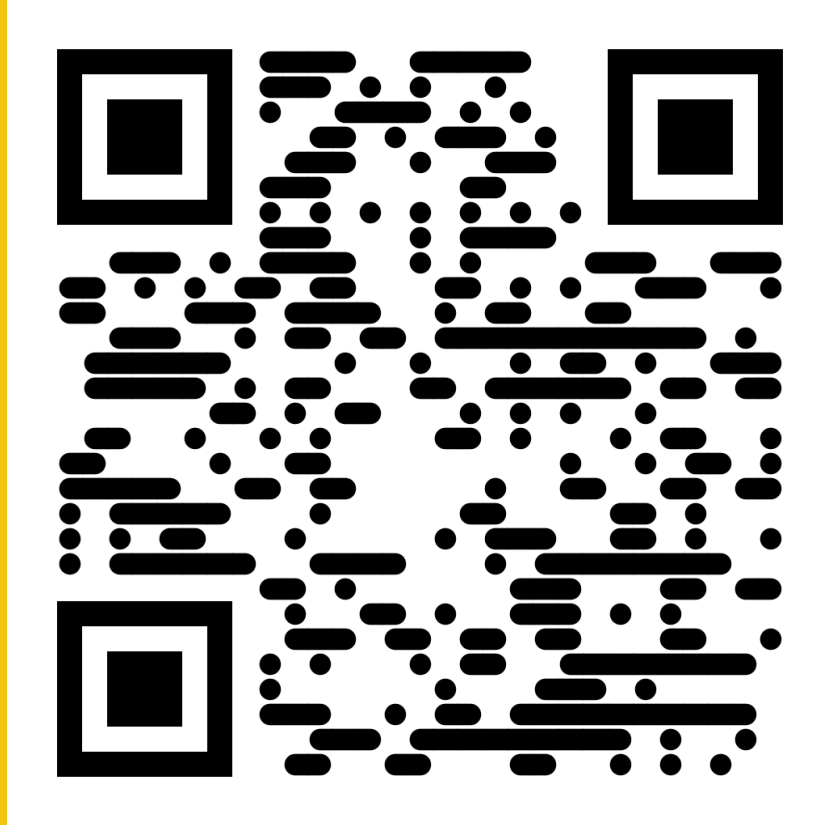
<https://absalon.ku.dk/enroll/8XAAWY>



Absalon



SLIDES:



Today's Menu

- Intros & Hello's
- Hands-on workshop with Arduino
- Ideation
- Project time
- Show, Talk, and Snacks

09:00	15m	Introduction	Introductions and the Hackathon Aims
09:15	1h 00m	Arduino 1 Instruction	Enter block description
10:15	15m	Break	Enter block description
10:30	45m	Arduino 2 Instruction	Enter block description
11:15	45m	Hackathon Intro, Team Formation	Enter block description
12:00	45m	Lunch	Enter block description
12:45	1h 00m	Work	Enter block description
13:45	15m	Break	Enter block description
14:00	1h 00m	Work and Refine	Enter block description
15:00	30m	Show, Talk, and Snacks	Enter block description
15:30			

“The philosophy behind Arduino is that if you want to learn electronics, you should be able to learn as you go from day one, instead of starting by learning algebra”

David Cuartielles

David Kushner (2011-10-26). "The Making of Arduino". IEEE Spectrum
<https://spectrum.ieee.org/geek-life/hands-on/the-making-of-arduino->



Scope of Arduino

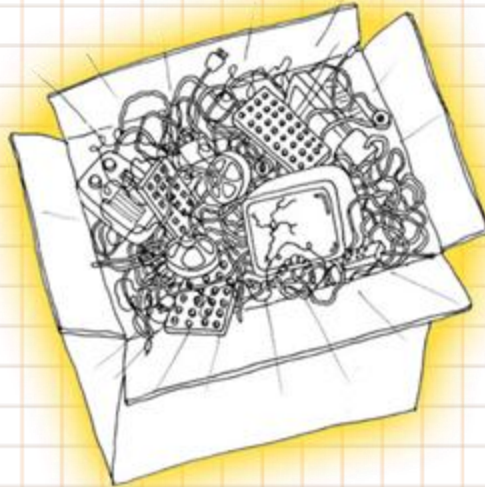


- Released in 2005 as a modest tool for Banzi's students at the Interaction Design Institute Ivrea (IDII)
- Join 433 244 hardware developers who follow Arduino on Hackster.io alone...
- Over a three month period in 2017, 42 million visits to the Arduino website came from recurring users—the die-hard makers.
- In June 2023 arduino.cc received 7.9M visits

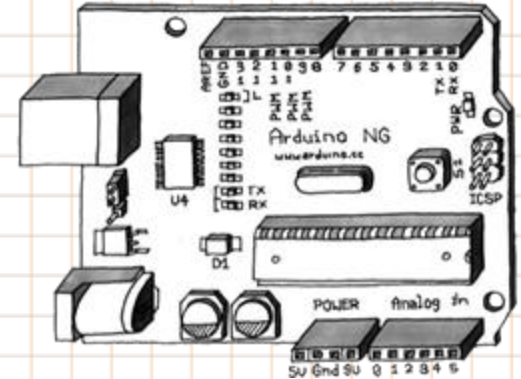
Feel it, Tinker



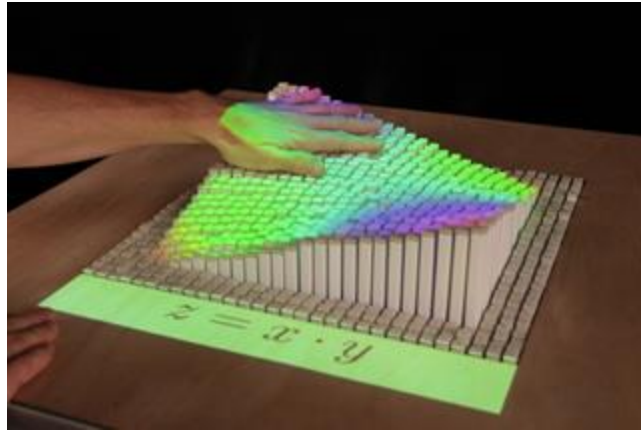
Junk is Gold



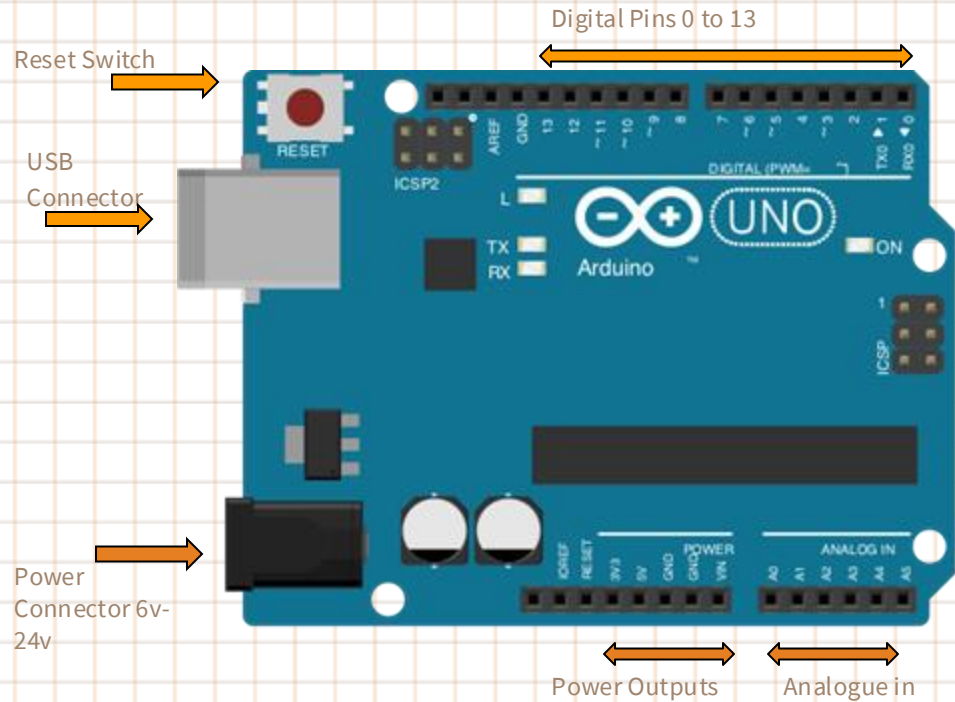
Arduino, what is that?



From Art to Science and to STEAM



The Pins



The Kit



Temperature

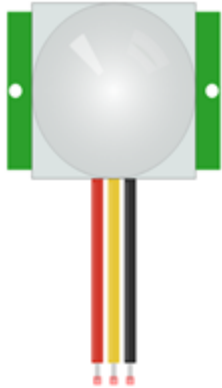


Button

Light Diode
Resistor



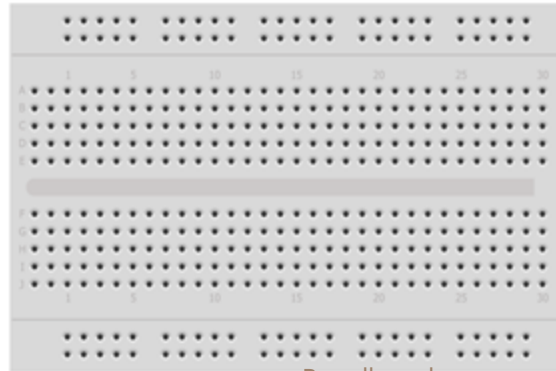
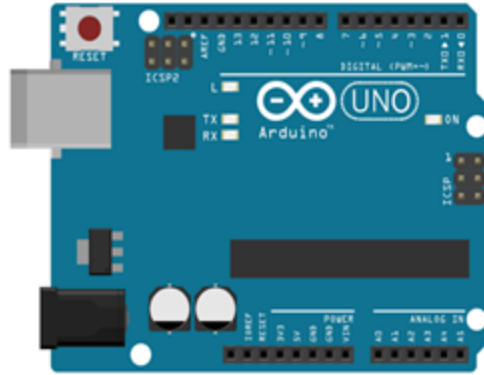
Potentiometer



PIR



Wires and Jumper



Breadboard



LEDs



Resistors



Piezo Speaker



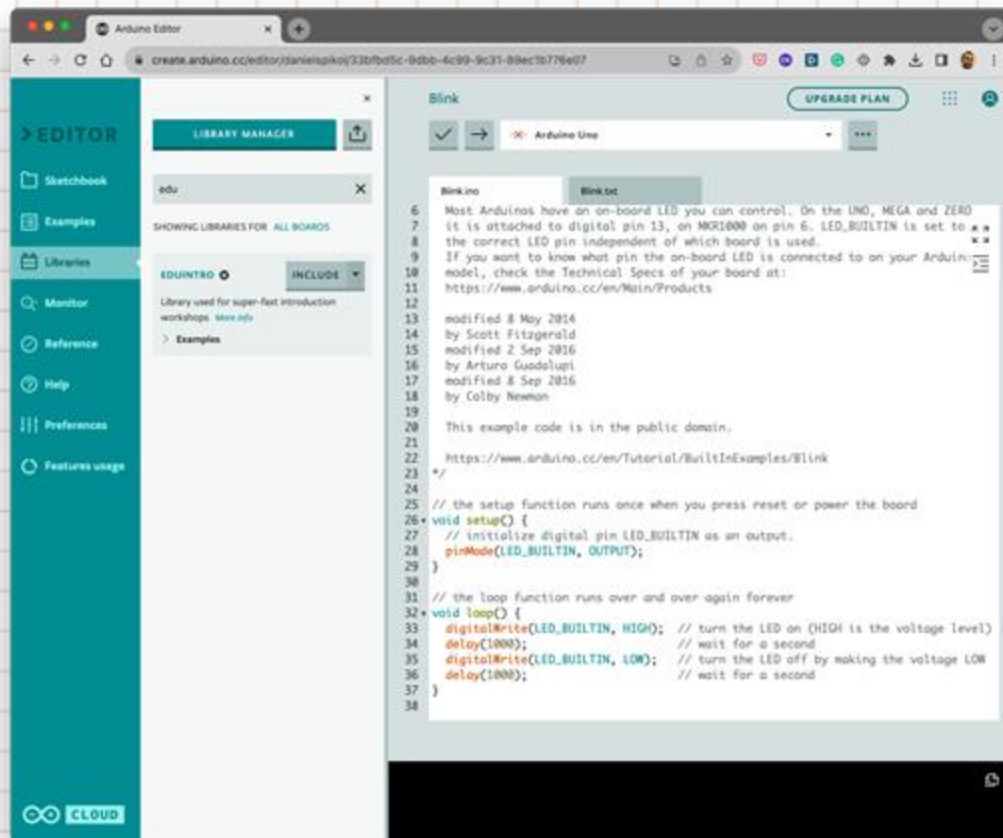
180 Servo & Continuous Servo

Use the IDE or Download the IDE

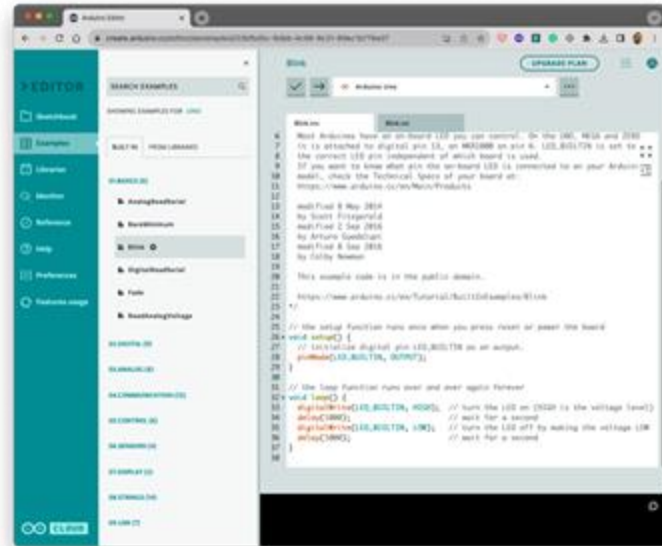
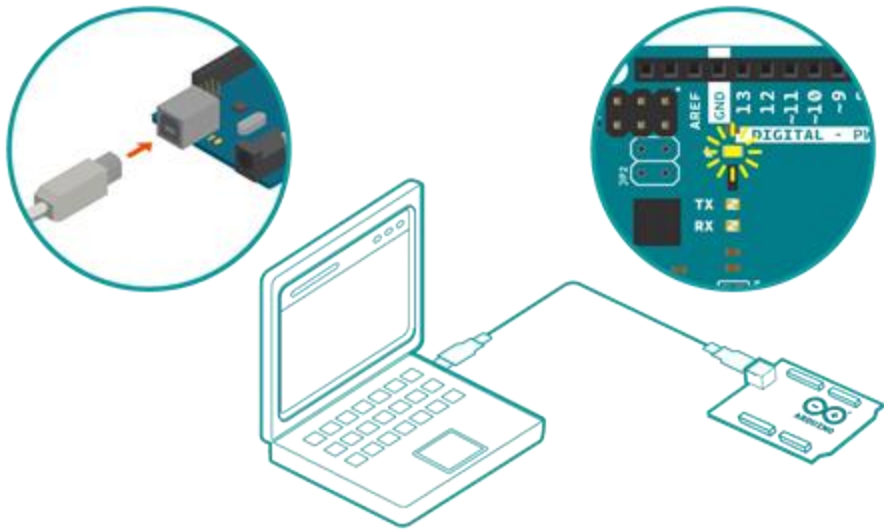
- Use the Web Editor or IDE from:
<https://www.arduino.cc>
- First let's blink
- Library needed also (next slides)



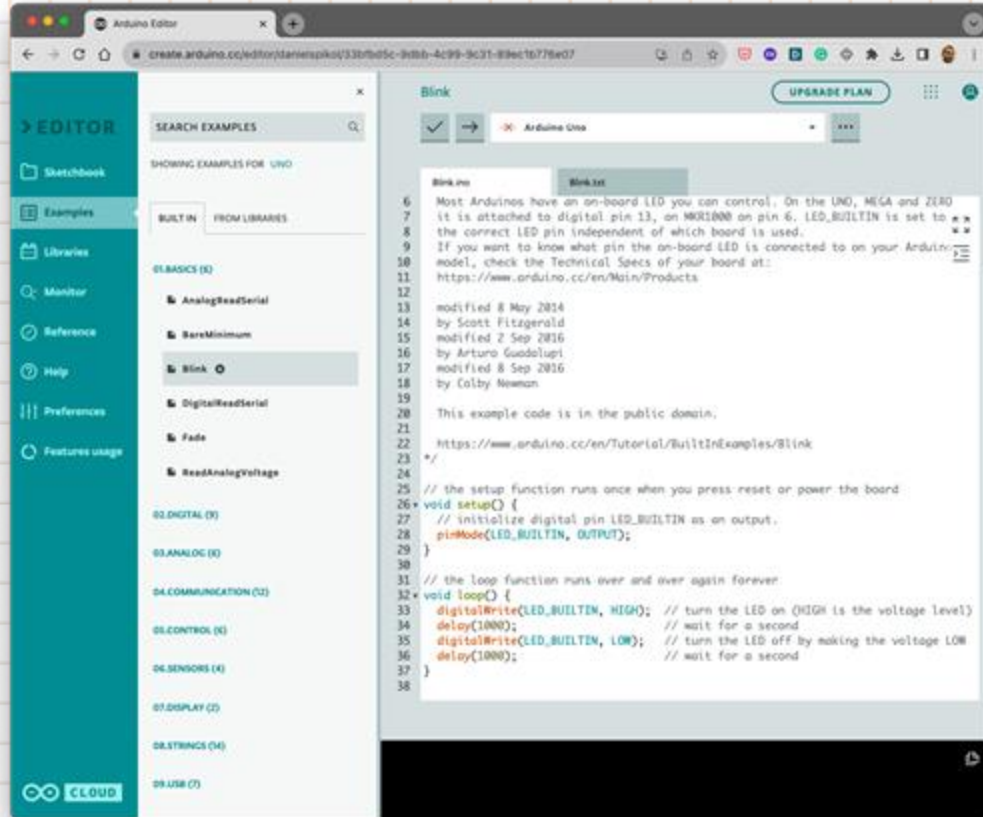
Arduino Web and EDUINTRO Library



Getting the Program to the Arduino Board



Arduino Web BLINK test



The IDE MENU

File Menu

New, Save, Open,
Examples...

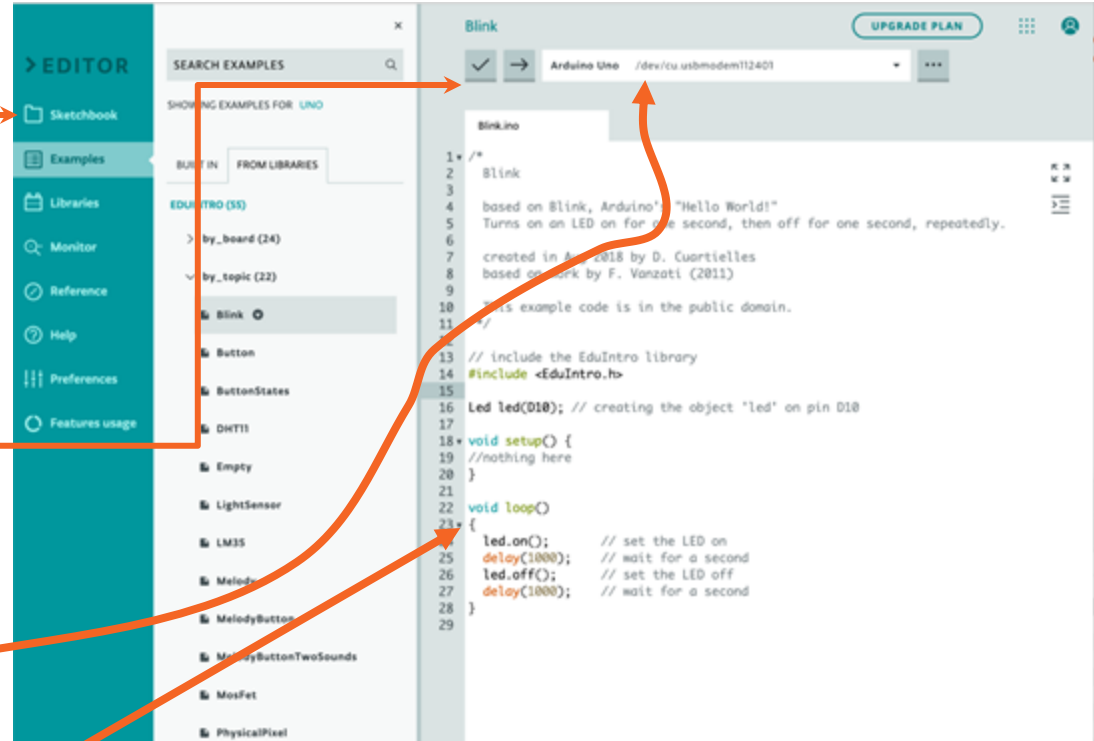
Verify and Upload

Test your code
Upload you code +

COM-Port Settings,
Tools, Board Type...

Code Area

Cut, Copy, Paste



Program structure

Declarations

Things the program needs to run.
These include extra libraries and
global variables and object

Set Up

Setup the method is used to initiate
the board before the code gets
executed.

Main Loop

It executes from top to bottom,
line by line, and then starts over
(looping back).

```
1 /*  
2  Blink  
3  
4  based on Blink, Arduino's "Hello World!"  
5  Turns on an LED on for one second, then off for one second, repeatedly.  
6  
7  created in Aug 2018 by D. Cuartielles  
8  based on work by F. Vanzati (2011)  
9  
10 This example code is in the public domain.  
11 */  
12  
13 // include the EduIntro library  
14 #include <EduIntro.h>  
15  
16 led led(D10); // creating the object 'led' on pin D10  
17  
18 void setup() {  
19   //nothing here  
20 }  
21  
22 void loop()  
23 {  
24   led.on();    // set the LED on  
25   delay(1000); // wait for a second  
26   led.off();   // set the LED off  
27   delay(1000); // wait for a second  
28 }  
29
```

A bit about the Arduino Language

C/C++ Based:

- The Arduino programming language is based on C/C++, which means it shares many of the same syntax rules, data types, and operators.

Setup and Loop Structure:

- Programs usually contain a void setup() and void loop() function. setup() runs once for initial settings, while loop() runs indefinitely.

Case Sensitivity:

- Arduino code is case-sensitive. For instance, Pin and pin would be treated as different variables.

Semicolons and Braces:

- Statements must be terminated with semicolons, and code blocks are encapsulated within curly braces {}.

Comments and Whitespace:

- Comments can be made with // for single-line comments or /* */ for multi-line comments. Whitespace is mostly ignored by the compiler but is useful for readability.

Code Structure

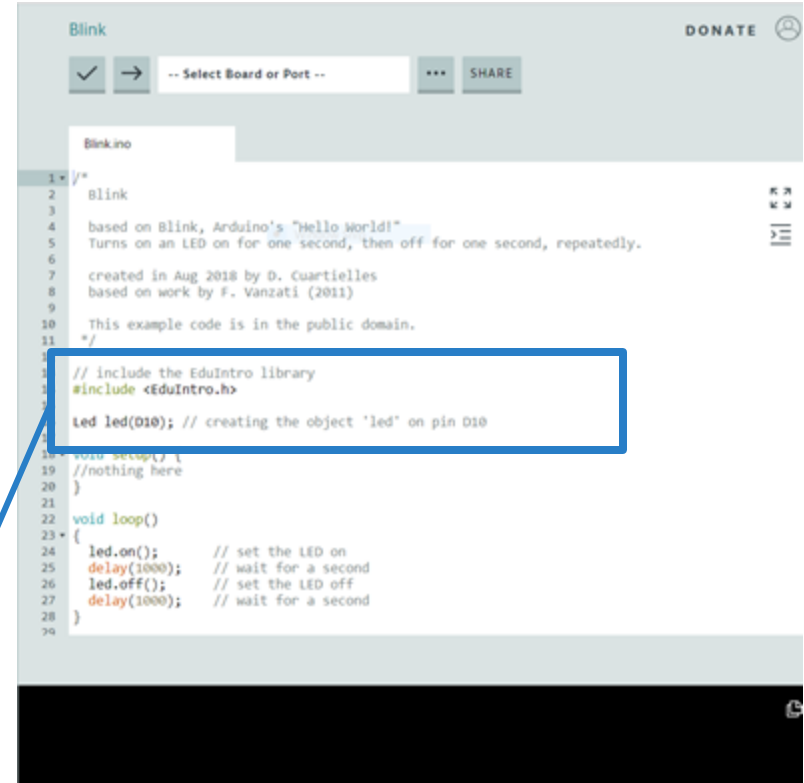
Includes & Variables

#Include is used to include outside libraries in your sketch. This gives the programmer access to a large group of extra C libraries (groups of pre-made functions). We will use the **EDUINTRO** Library this week

Variables are **containers for values**.

You can give a variable any name you like.

The **value** of a variable can be changed and used dynamically as many times as you like.



```
1 /*
2  * Blink
3  *
4  * based on Blink, Arduino's "Hello World!"
5  * Turns on an LED on for one second, then off for one second, repeatedly.
6  *
7  * created in Aug 2018 by D. Cuartielles
8  * based on work by F. Vanzati (2011)
9  *
10  * This example code is in the public domain.
11  */
12
13 // include the EduIntro library
14 #include <EduIntro.h>
15
16 Led led(D10); // creating the object 'led' on pin D10
17
18 void setup() {
19   //nothing here
20 }
21
22 void loop()
23 {
24   led.on(); // set the LED on
25   delay(1000); // wait for a second
26   led.off(); // set the LED off
27   delay(1000); // wait for a second
28 }
29
```

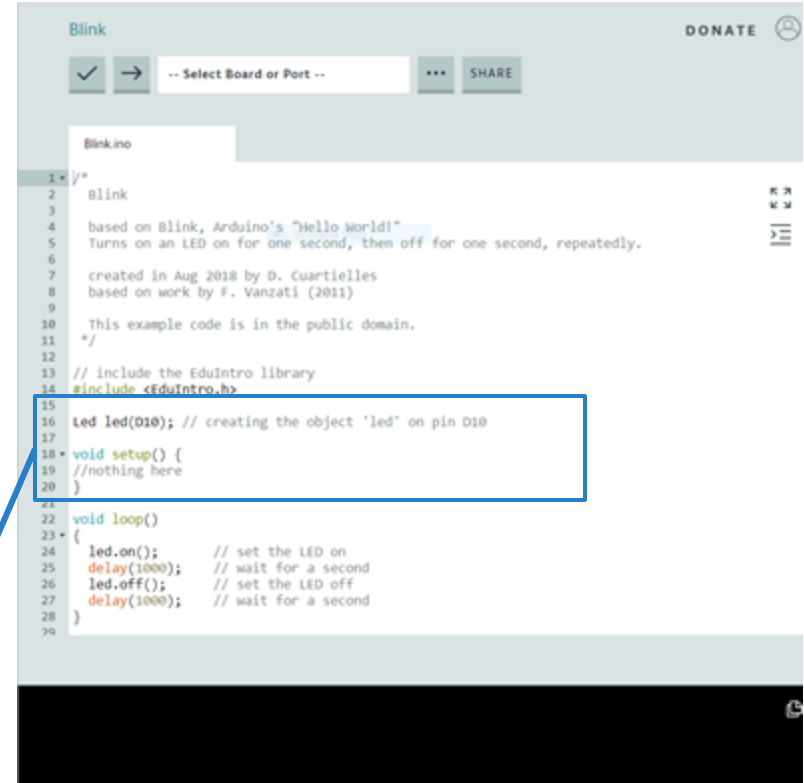
Code Structure

The Setup Method

Setup the method is used to initiate the board before the code gets executed.

This code runs only once when the reset button has been pressed.

It defines if pins should be used **as inputs or outputs**, **whether serial communication** is going to be used, etc.



```
1 /*  
2  Blink  
3  
4  based on Blink, Arduino's "Hello World!"  
5  Turns on an LED on for one second, then off for one second, repeatedly.  
6  
7  created in Aug 2018 by D. Cuartielles  
8  based on work by F. Vanzati (2011)  
9  
10 This example code is in the public domain.  
11 */  
12  
13 // include the EduIntro library  
14 #include <EduIntro.h>  
15  
16 led led(D10); // creating the object 'led' on pin D10  
17  
18 void setup() {  
19   //nothing here  
20 }  
21  
22 void loop()  
23 {  
24   led.on();           // set the LED on  
25   delay(1000);        // wait for a second  
26   led.off();          // set the LED off  
27   delay(1000);        // wait for a second  
28 }  
29
```

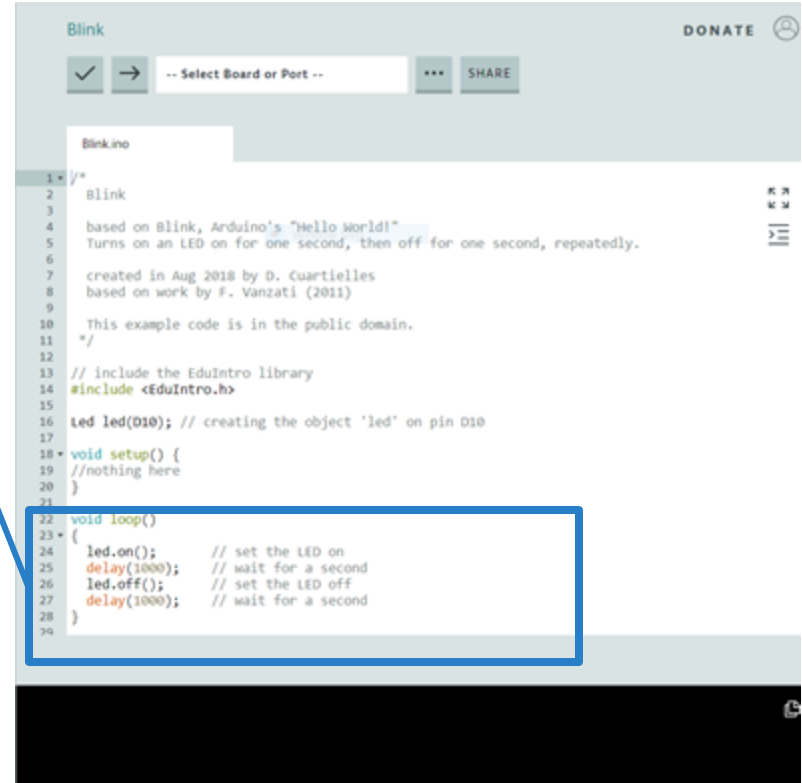
Code Structure

The Program Loop

The loop is where the action is!

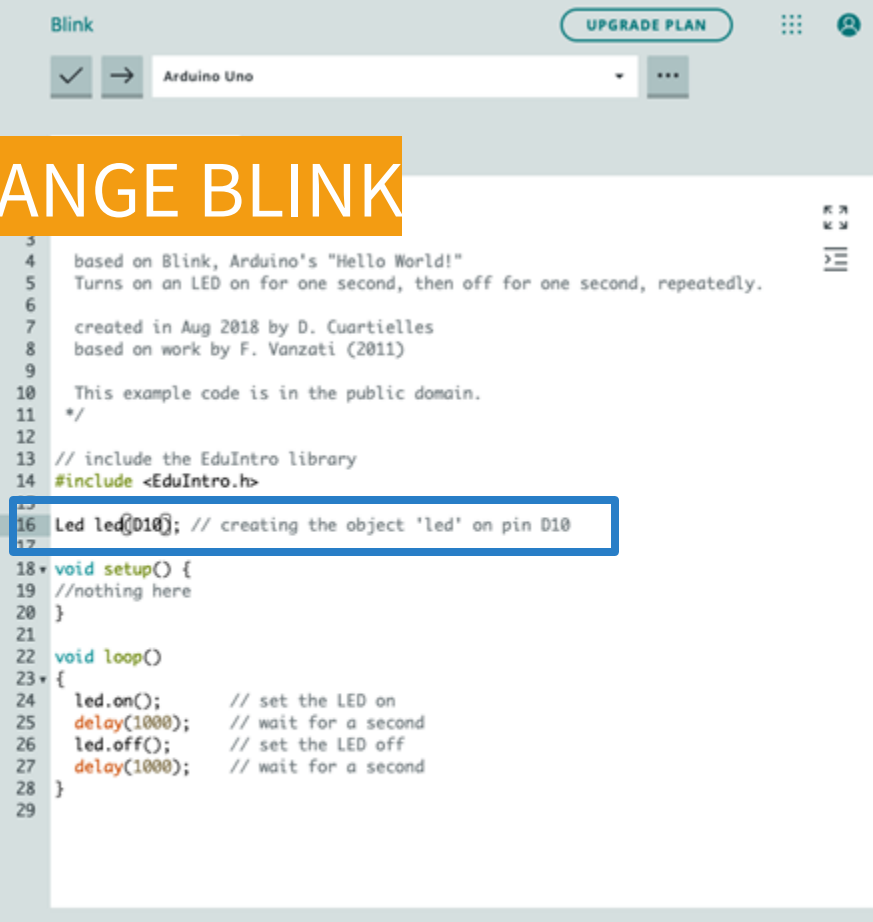
This is where you write your program

It executes from **top to bottom**, line by line, and then starts over (**looping back**).



```
1 /*  
2  Blink  
3  
4  based on Blink, Arduino's "Hello World!"  
5  Turns on an LED on for one second, then off for one second, repeatedly.  
6  
7  created in Aug 2018 by D. Cuartielles  
8  based on work by F. Vanzati (2011)  
9  
10 This example code is in the public domain.  
11 */  
12  
13 // include the EduIntro library  
14 #include <EduIntro.h>  
15  
16 led led(D10); // creating the object 'led' on pin D10  
17  
18 void setup() {  
19   //nothing here  
20 }  
21  
22 void loop()  
23 {  
24   led.on();           // set the LED on  
25   delay(1000);        // wait for a second  
26   led.off();          // set the LED off  
27   delay(1000);        // wait for a second  
28 }  
29
```

Take a look and CHANGE BLINK



```
Blink
UPGRADE PLAN
✓ → Arduino Uno ...

3
4 based on Blink, Arduino's "Hello World!"
5 Turns on an LED on for one second, then off for one second, repeatedly.
6
7 created in Aug 2018 by D. Cuartielles
8 based on work by F. Vanzati (2011)
9
10 This example code is in the public domain.
11 */
12
13 // include the EduIntro library
14 #include <EduIntro.h>
15
16 Led led(D10); // creating the object 'led' on pin D10
17
18 void setup() {
19 //nothing here
20 }
21
22 void loop()
23 {
24 led.on(); // set the LED on
25 delay(1000); // wait for a second
26 led.off(); // set the LED off
27 delay(1000); // wait for a second
28 }
29
```

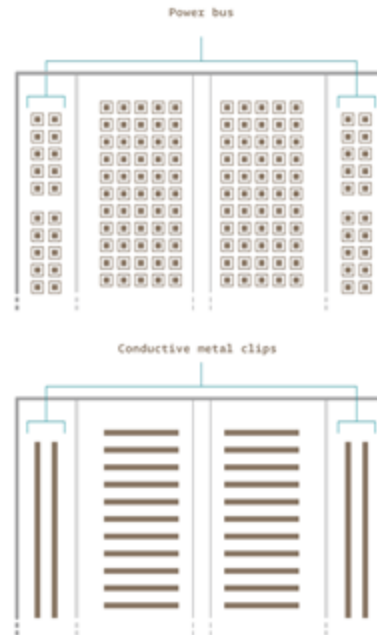
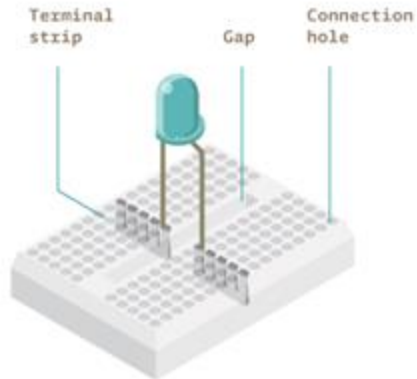
EDUINTRO Examples BLINK

Let's add an Output - LED

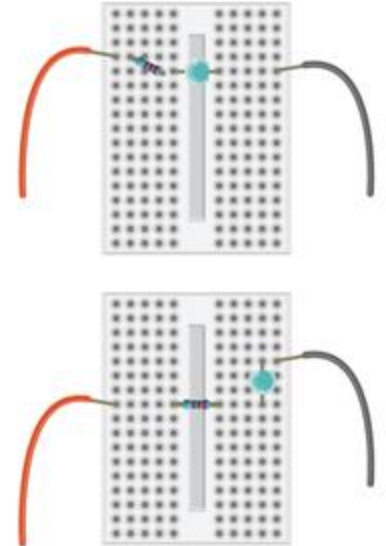


More about Breadboards

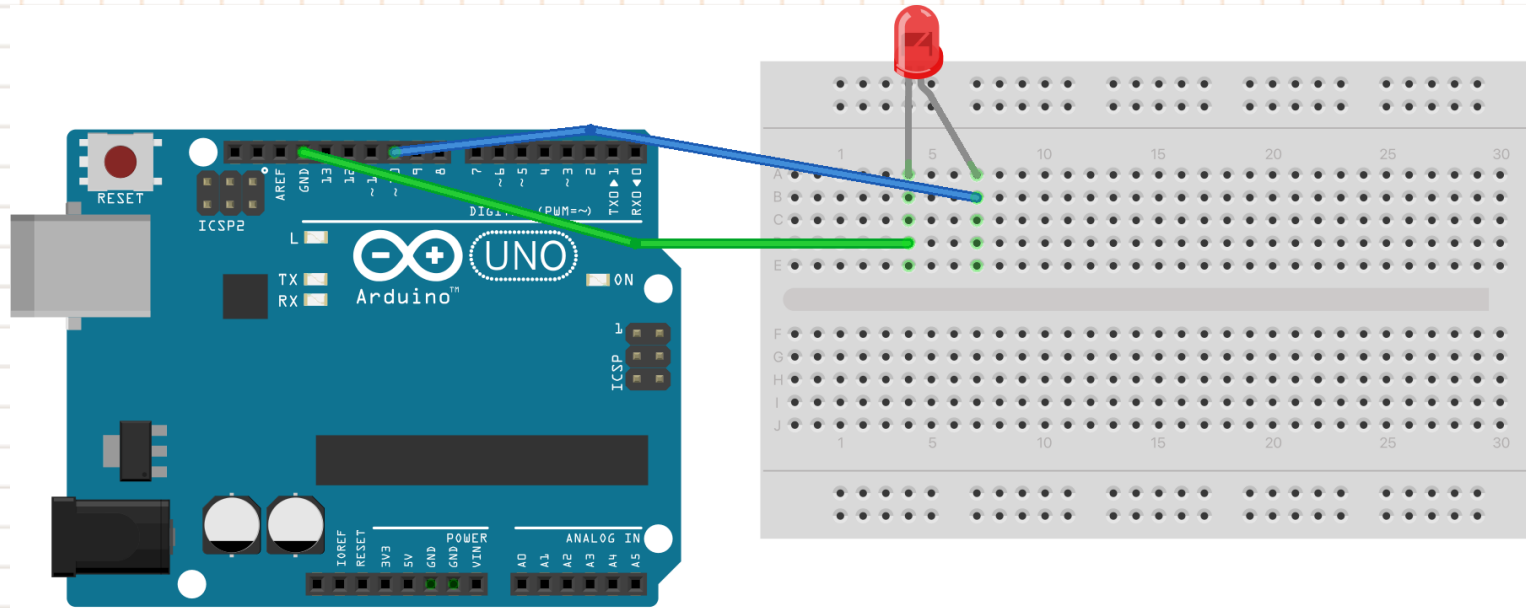
INSIDE A BREADBOARD



SIMPLE BREADBOARD CIRCUITS



Wire up the LED

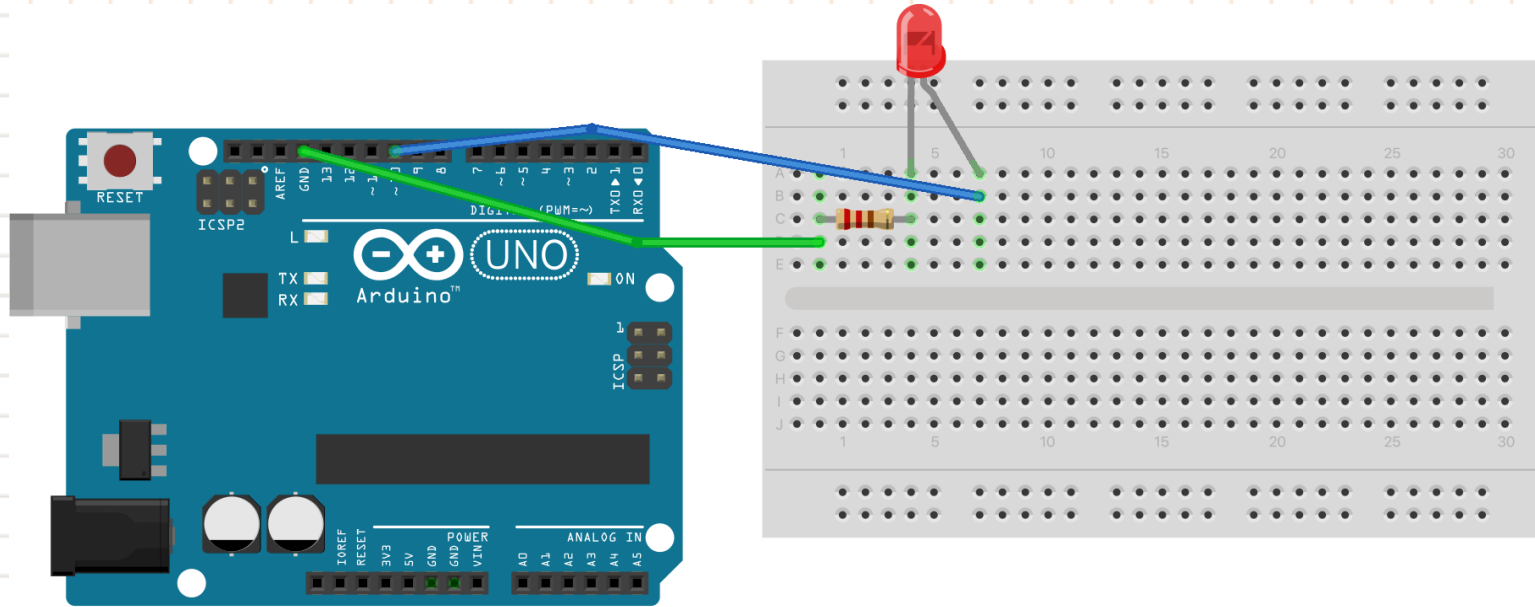


fritzing

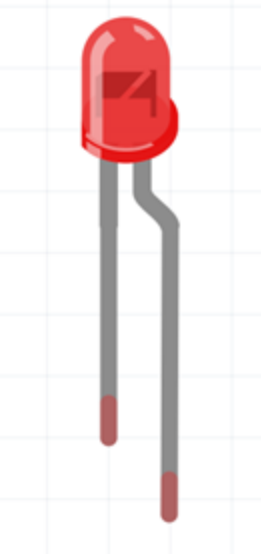
Long side of
LED is
+positive

EDUINTRO Examples BLINK

Wire up the LED + Resistor



fritzing



Long side of
LED is
+positive

EDUINTRO Examples BLINK

Take a look and CHANGE BLINK

```
Blink
UPGRADE PLAN
✓ → Arduino Uno
3
4 based on Blink, Arduino's "Hello World!"
5 Turns on an LED on for one second, then off for one second, repeatedly.
6
7 created in Aug 2018 by D. Cuartielles
8 based on work by F. Vanzati (2011)
9
10 This example code is in the public domain.
11 */
12
13 // include the EduIntro library
14 #include <EduIntro.h>
15
16 Led led(D10); // creating the object 'led' on pin D10
17
18 void setup() {
19 //nothing here
20 }
21
22 void loop()
23 {
24 led.on(); // set the LED on
25 delay(1000); // wait for a second
26 led.off(); // set the LED off
27 delay(1000); // wait for a second
28 }
29
```

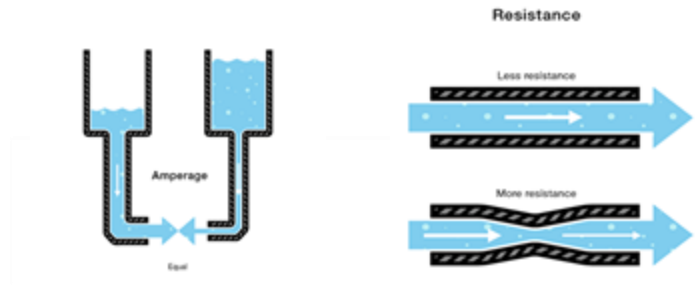
EDUINTRO Examples BLINK

Resistors

Current (A): is the number of electrons per time and section of cable

Voltage (V): is the difference in the number of electrons between two points, It only exists in relative terms!, thus we have to “share grounds” when interconnecting circuits

Resistance (R): reduces the flow of electrons



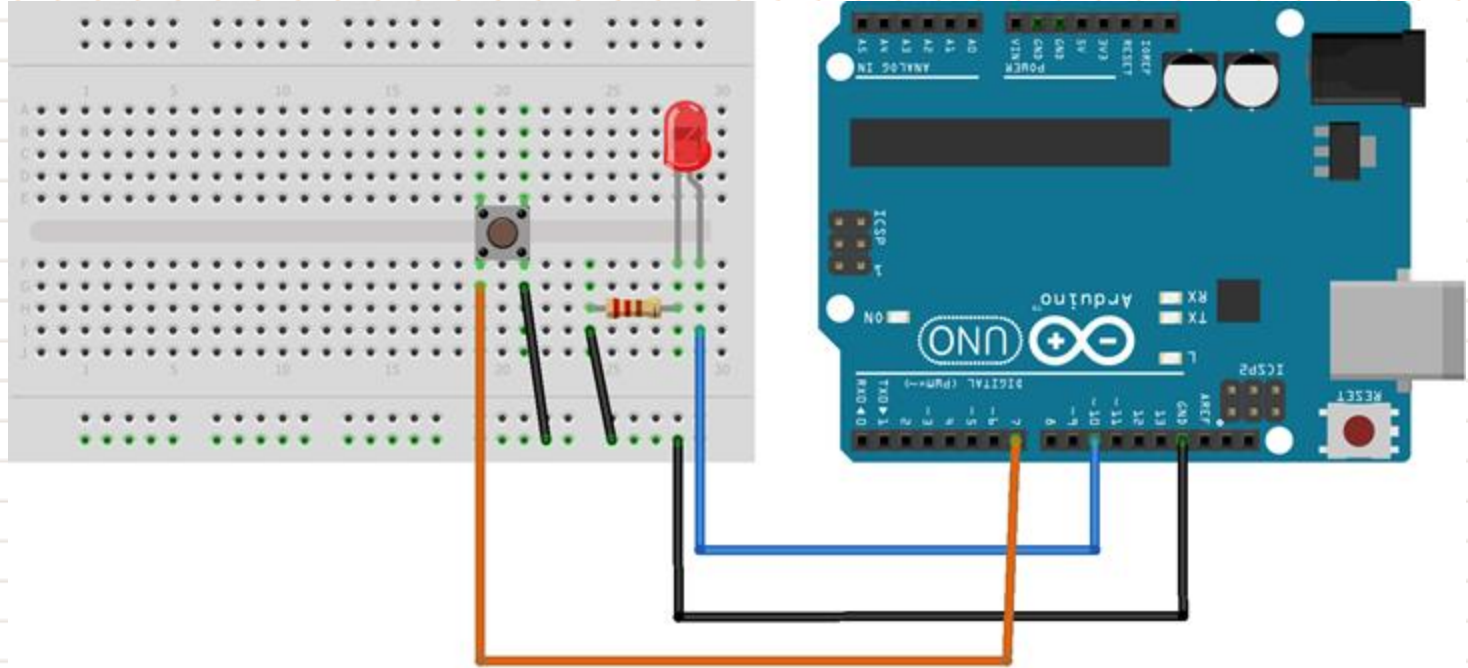
Current always takes the easiest possible way

- If it has the option to pass through cable or resistor, it chooses the cable

Now add Button for the LED



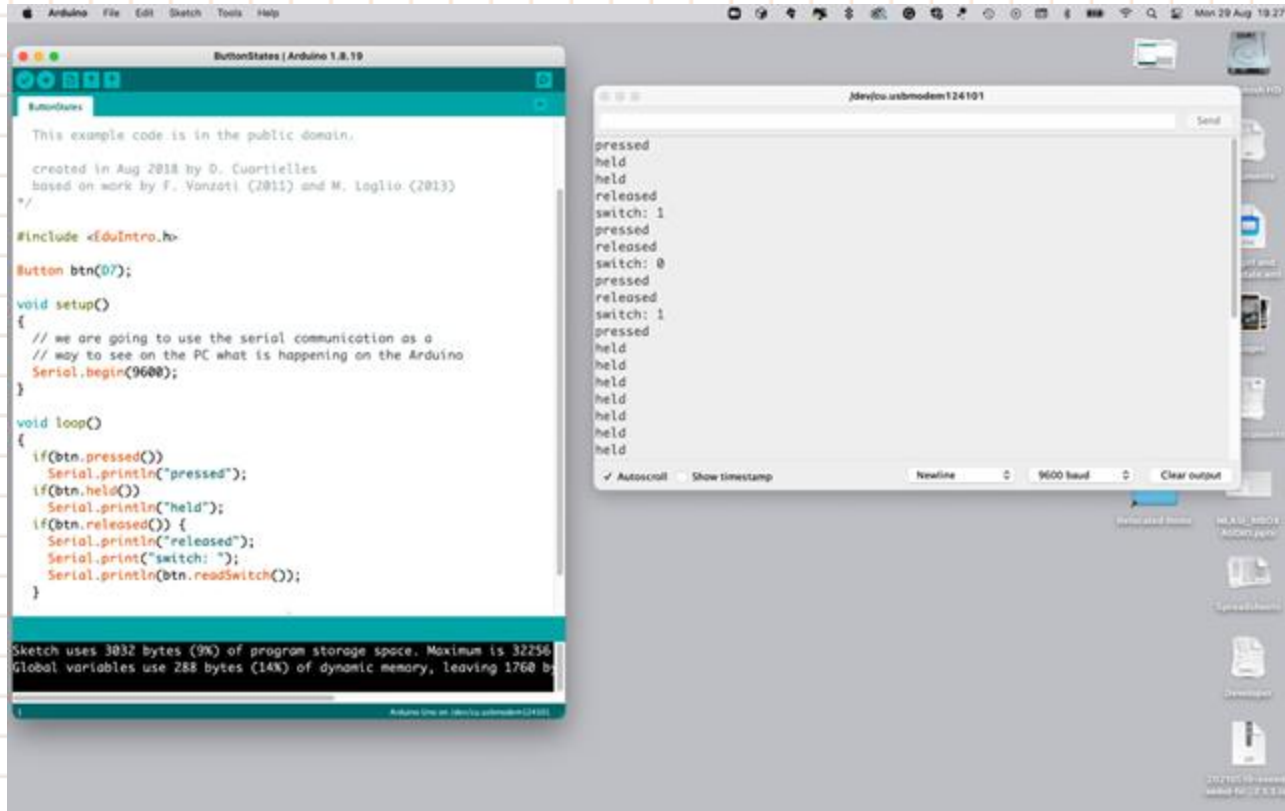
Long side of
LED is +positive



Code EDUINTRO Button

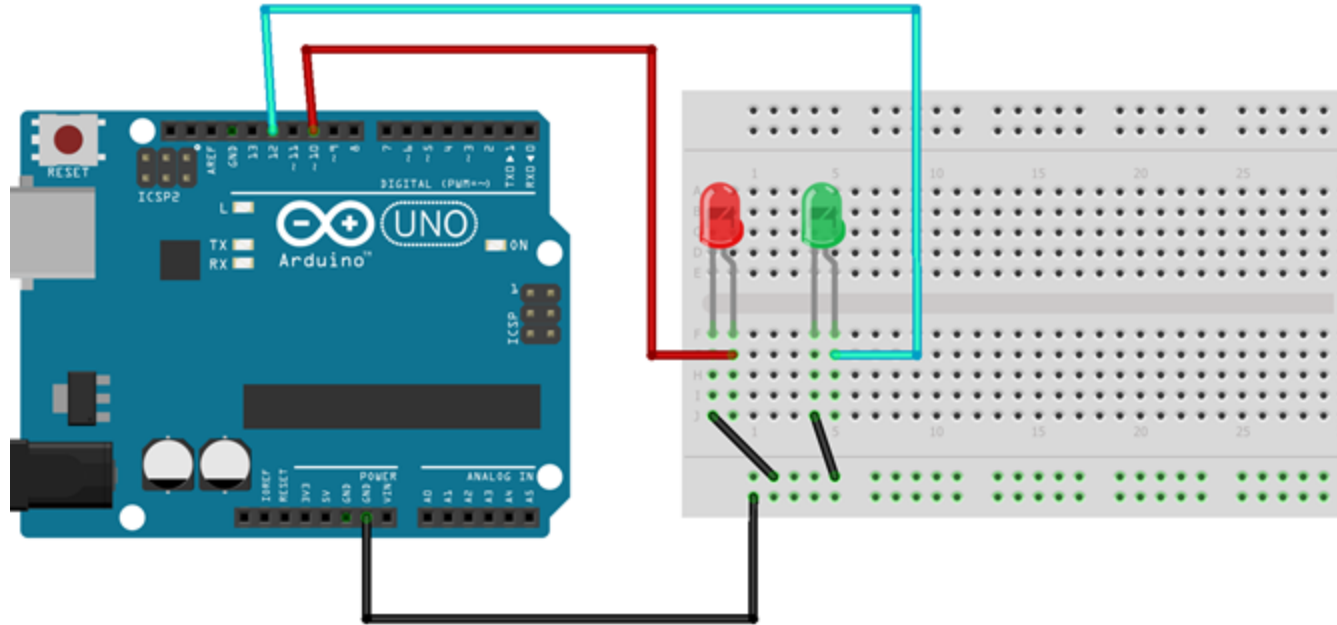
fritzing

More about Buttons



Code EDUINTRO Button

Double the fun & the code!



fritzi

Double the fun & the code!

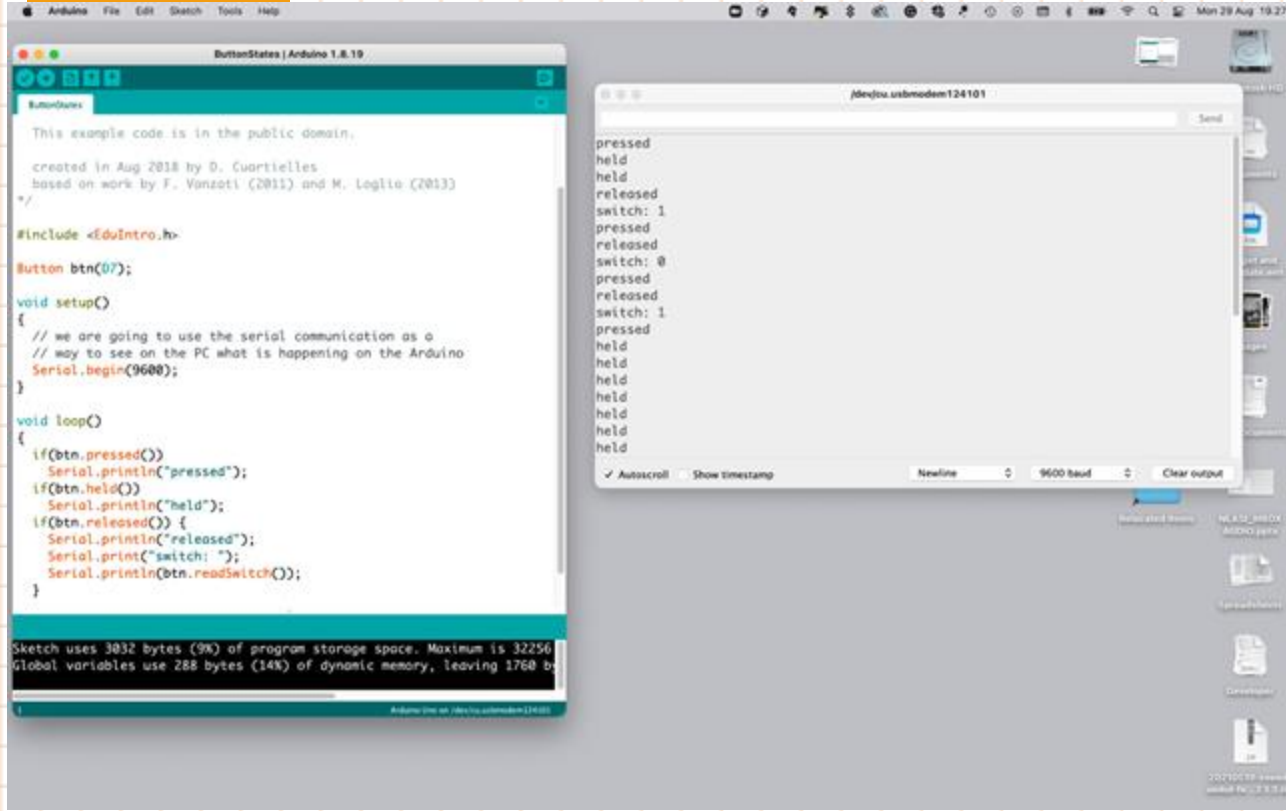
New LED object
At D11

Copy, Paste,
and play around!

```
Day1_Blink_LEDs_seque
14 // include the Arduino library
15 #include <EduIntro.h>
16 Led led1(D10); // creating the object 'led' on pin D10
17 Led led2(D11); // creating the object 'led' on pin D11
18
19 void setup() {
20 //nothing here
21 }
22
23 void loop()
24 {
25     led1.on(); // set the LED1 on
26     delay(1000); // wait for a second
27     led1.off(); // set the LED1 off
28     delay(1000); // wait for a second
29
30     led2.on(); // set the LED2 on
31     delay(1000); // wait for a second
32     led2.off(); // set the LED2 off
33     delay(1000); // wait for a second
34
35     led1.on(); // set the LED1 on
36     led2.on(); // set the LED2 on
37     delay(1000); // wait for a second
38     led1.off(); // set the LED1 off
39     led2.off(); // set the LED2 off
40     delay(1000); // wait for a second
41 }
42
```

Button States and different LEDs If ... then

More about Buttons and Serial



Code EDUINTRO ButtonState

PIEZO “Pump up the Volume”

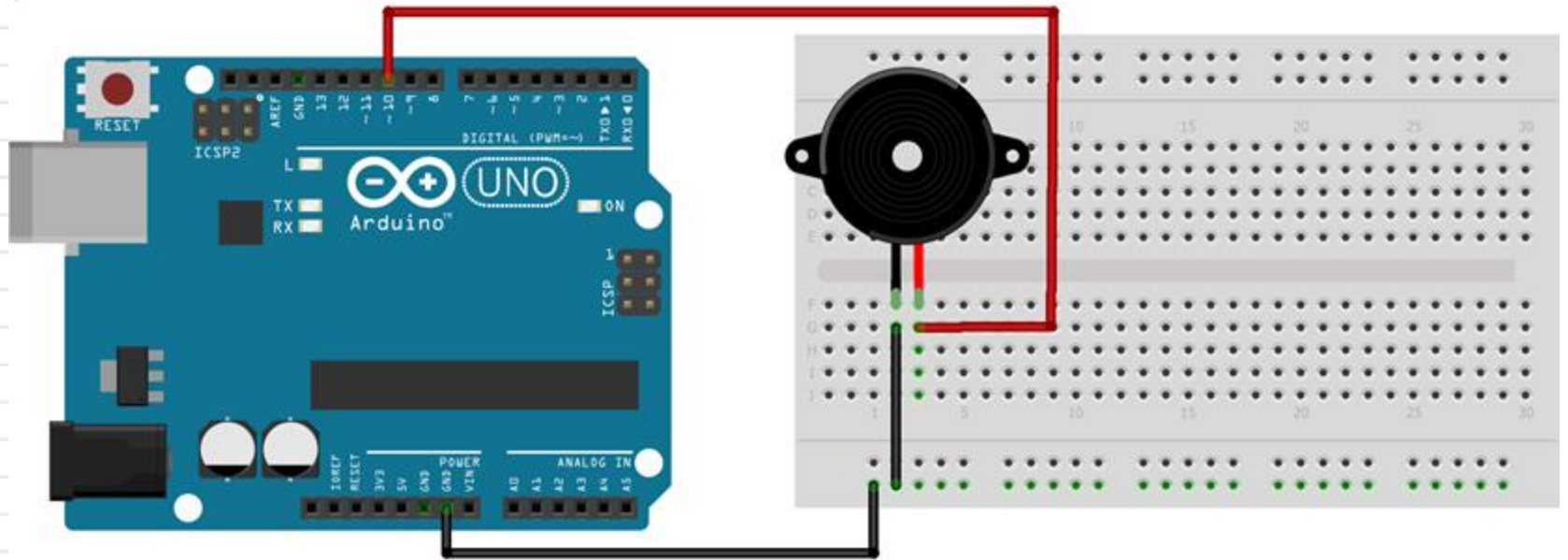
Basic Piezo Concepts

Array variables see Ln 17 `int melody[] = {`

```
NOTE_C4, 4,  
NOTE_G3, 8,  
NOTE_G3, 8,  
NOTE_A3, 4,  
NOTE_G3, 4,  
SILENCE, 4,  
NOTE_B3, 4,  
NOTE_C4, 4 };
```

```
Melody.ino  
  
8  
9   This example code is in the public domain.  
10  */  
11  
12  // include the EduIntro library  
13  #include <EduIntro.h>  
14  
15  // define the song as (note, duration) pairs  
16  // note durations: 4 = quarter note, 8 = eighth note, etc.:  
17  int melody[] = { NOTE_C4, 4,  
18                  NOTE_G3, 8,  
19                  NOTE_G3, 8,  
20                  NOTE_A3, 4,  
21                  NOTE_G3, 4,  
22                  SILENCE, 4,  
23                  NOTE_B3, 4,  
24                  NOTE_C4, 4 };  
25  
26  Piezo piezo(D10); // creating the object 'piezo' on pin D10  
27  
28  void setup() {  
29    //nothing here  
30  }  
31  
32  void loop()  
33  {  
34    piezo.play(melody); // play the song  
35    delay(1000);        // wait for a second  
36  }  
37
```

Piezo Concepts



fritzing

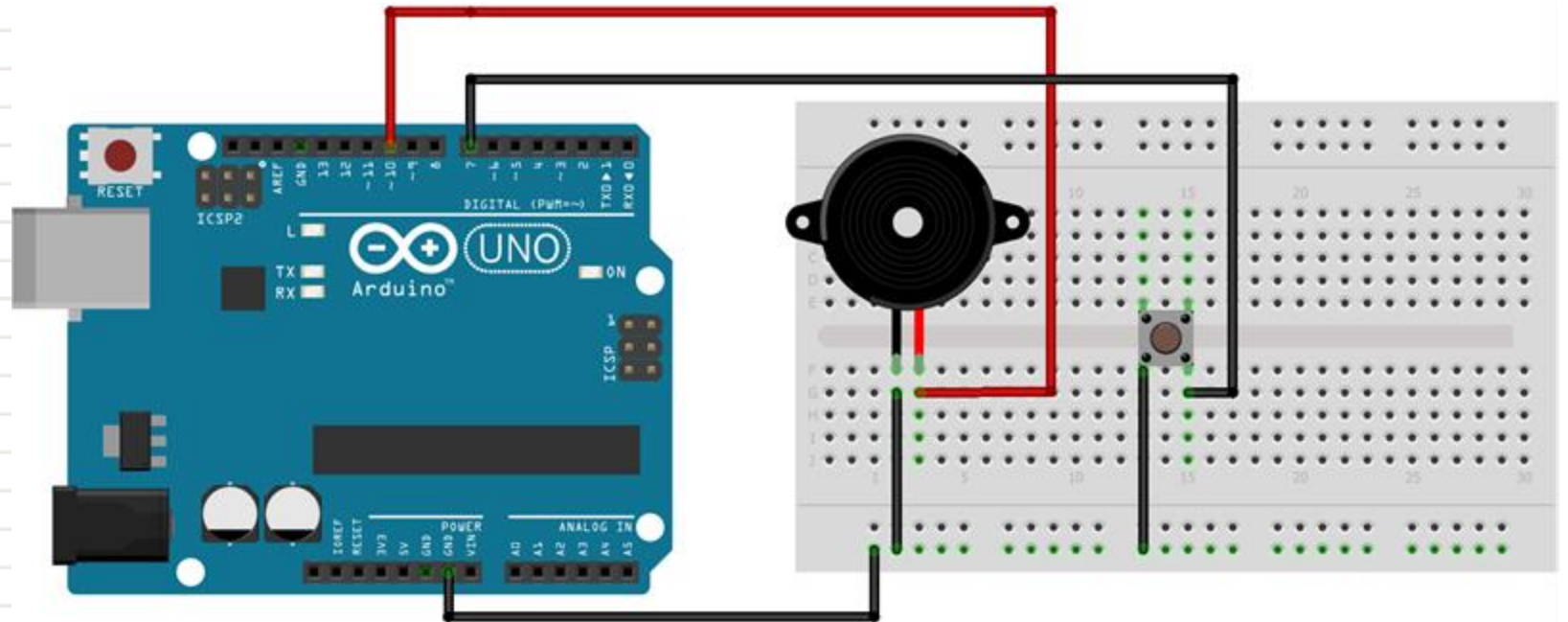
Piezo and Button

- Concepts
 - Array
 - Conditional Statements

EduIntro >>courseware>>workshop_1h >>
MelodyButton

```
MelodyButton.ino
12 // include the EduIntro library
13 #include <EduIntro.h>
14
15 // define the song as (note, duration) pairs
16 // note durations: 4 = quarter note, 8 = eighth note, etc.:
17 int melody[] = { NOTE_C4, 4,
18                  NOTE_G3, 8,
19                  NOTE_G3, 8,
20                  NOTE_A3, 4,
21                  NOTE_G3, 4,
22                  SILENCE, 4,
23                  NOTE_B3, 4,
24                  NOTE_C4, 4 };
25
26 Button button(D7); // creating the object 'button' on pin D7
27
28 Piezo piezo(D10); // creating the object 'piezo' on pin D10
29
30 void setup() {
31   //nothing here
32 }
33
34 void loop()
35 {
36   // if the button was just pressed, play melody
37   if (button.pressed()) {
38     piezo.play(melody); // play the song
39   }
40 }
41
```

Piezo and the Button



fritzing

Servo Motors

Servo Motors

Small = 180 direction

Big = continuous

Brown=ground

Red = 5V

Yell = signal

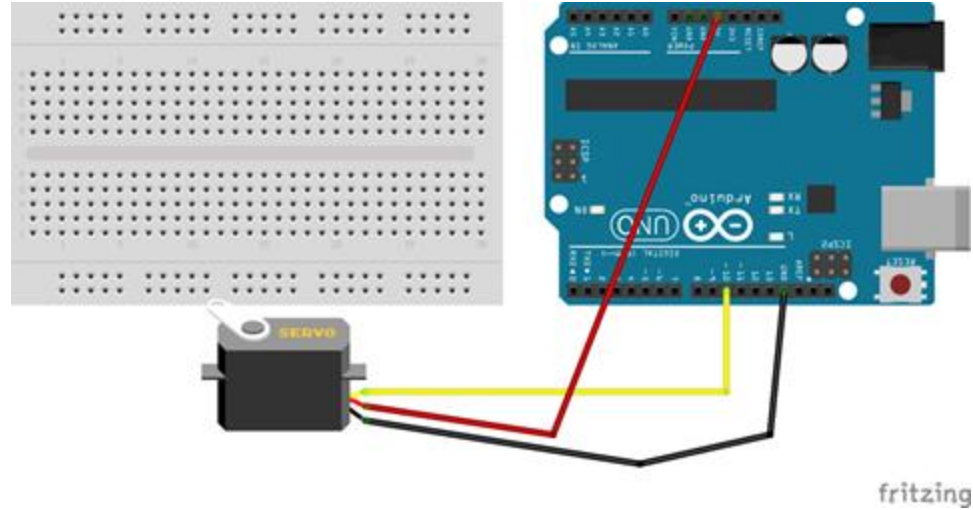


180 Sweep

- Ingredients
 - 180 Servo
- Cooking

EduIntro

>>coursware>>workshop_1h>>Servo



180 Sweep

- Ingredients
 - 180 Servo
- Cooking

EduIntro

>>coursware>>workshop_1h>>Servo



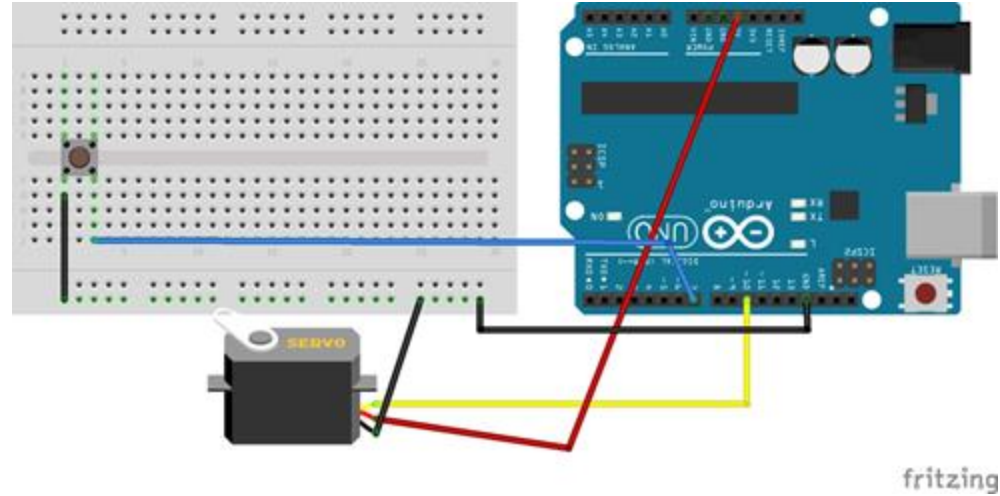
```
Servo
UPGRADE PLAN
✓ → Arduino Uno
Servo.ino
1 /*
2  Servo
3
4  Makes a Servo change angle after some time.
5
6  This example code is in the public domain.
7
8  created in Aug 2018 by D. Cuartielles
9
10 based on http://www.arduino.cc/en/Tutorial/Button
11 */
12
13 // include the EduIntro library
14 #include <EduIntro.h>
15
16 ServoMotor servo(D10); // creating the object 'servo' on pin D10
17
18
19 void setup() {
20 //nothing here
21 }
22
23 void loop()
24 {
25   servo.write(90);
26   delay (1000); // wait for a second
27   servo.write(0);
28   delay (1000); // wait for a second
29 }
30
```

180 Button

- Ingredients
 - 180 Servo
 - Button
- Cooking

EduIntro

>>coursware>>workshop_1h>>Serv
oButton



180 Button

- Ingredients
 - 180 Servo
 - Button
- Cooking

EduIntro

>>courseware>>workshop_1h>>ServoButton

180 Button

- Concept
 - Explore what happens when you switch the value to 180 and other values even 200
- Cooking

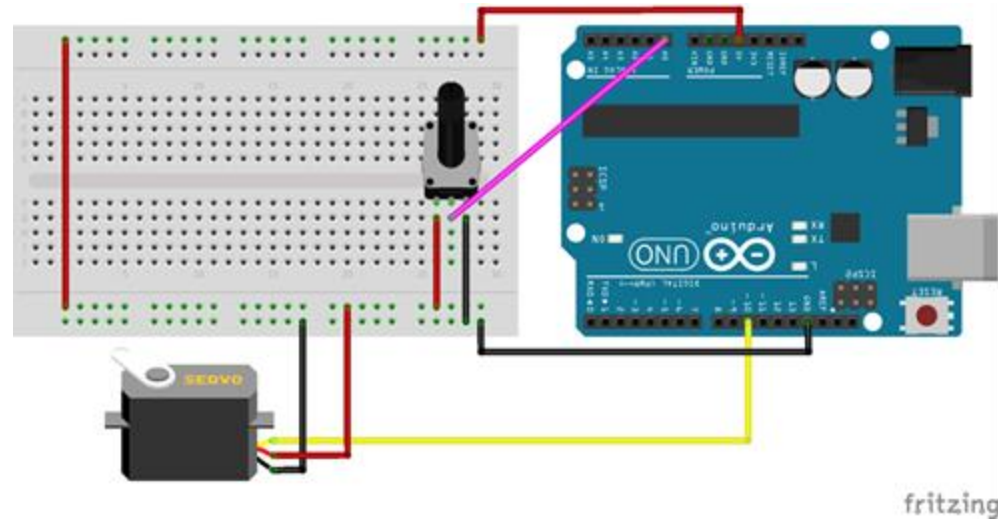
EduIntro

>>courseware>>workshop_1h>>Serv
oButton

```
void loop()
{
  servo.write(90);
  delay (1000);    // wait for a second
  servo.write(0);
  delay (1000);    // wait for a second
}
```

Continuous Motor Potentiometer

- Ingredients
 - Big Servo Motor
 - Potentiometer
- Cooking
 - EduIntro >>by_topic>>
ServoKnobContinuous



Continuous Motot Potentiometer

- Concepts
 - Continuous motors
 - 90 is stop
 - 0-89 clockwise with speed
 - 91-179 counterclockwise with speed

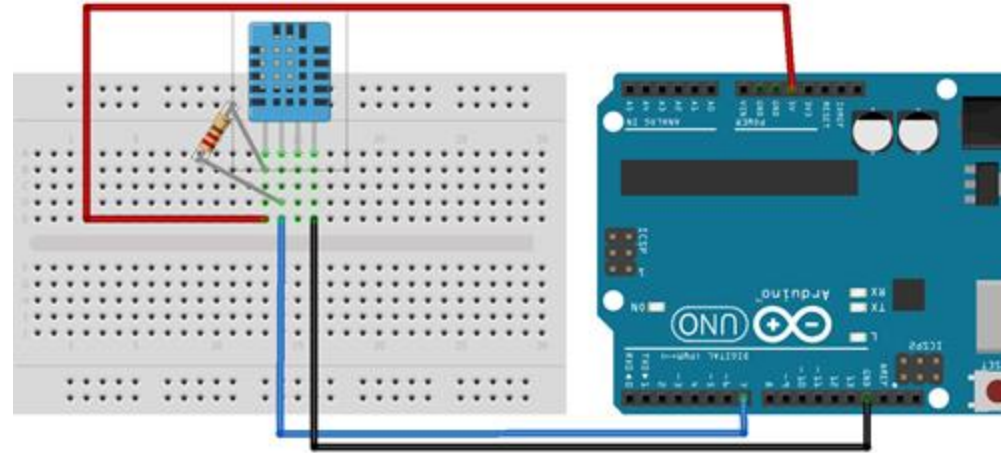
```
12 // include the EduIntro library
13 #include <EduIntro.h>
14
15 Potentiometer pot(A0); // creating the object 'pot' on pin A0
16
17 ServoMotor servo(D10); // creating the object 'servo' on pin D10
18
19 int potVal = 0; // value read from the pot
20 int angleVal = 0; // value of the angle to set the servo
21
22 void setup() {
23   // initialize serial communications at 9600 bps
24   Serial.begin(9600);
25 }
26
27 void loop()
28 {
29   // read the potentiometer's value:
30   potVal = pot.read();
31
32   // map the data from the sensor (0..1023) to
33   // one the servo can take (0..180)
34   angleVal = map(potVal, 0, 1023, 0, 180);
35
36   // set the led brightness
37   servo.write(angleVal);
38
39   // Signify it different events for the motor
40   if (angleVal == 90)
41     Serial.println("Motor Stop");
42   if (angleVal <= 1)
43     Serial.println("Motor Max clockwise");
44   if (angleVal >= 179)
45     Serial.println("Motor Max counter clockwise");
46
47   // print the results to the serial monitor:
48   Serial.print("angle = ");
49   Serial.println(angleVal);
50
51   // wait 100 milliseconds before the next loop
52   delay(100);
53 }
54
55
```

Digital Humidity and Temperature DHT11 Sensor



DHT11

- Cooking
 - DHT11 Sensor
 - Resistor (in bag)
 - Between 5V and Signal
- Ingredients
 - EDUINTRO>>by_topic>>DHT11

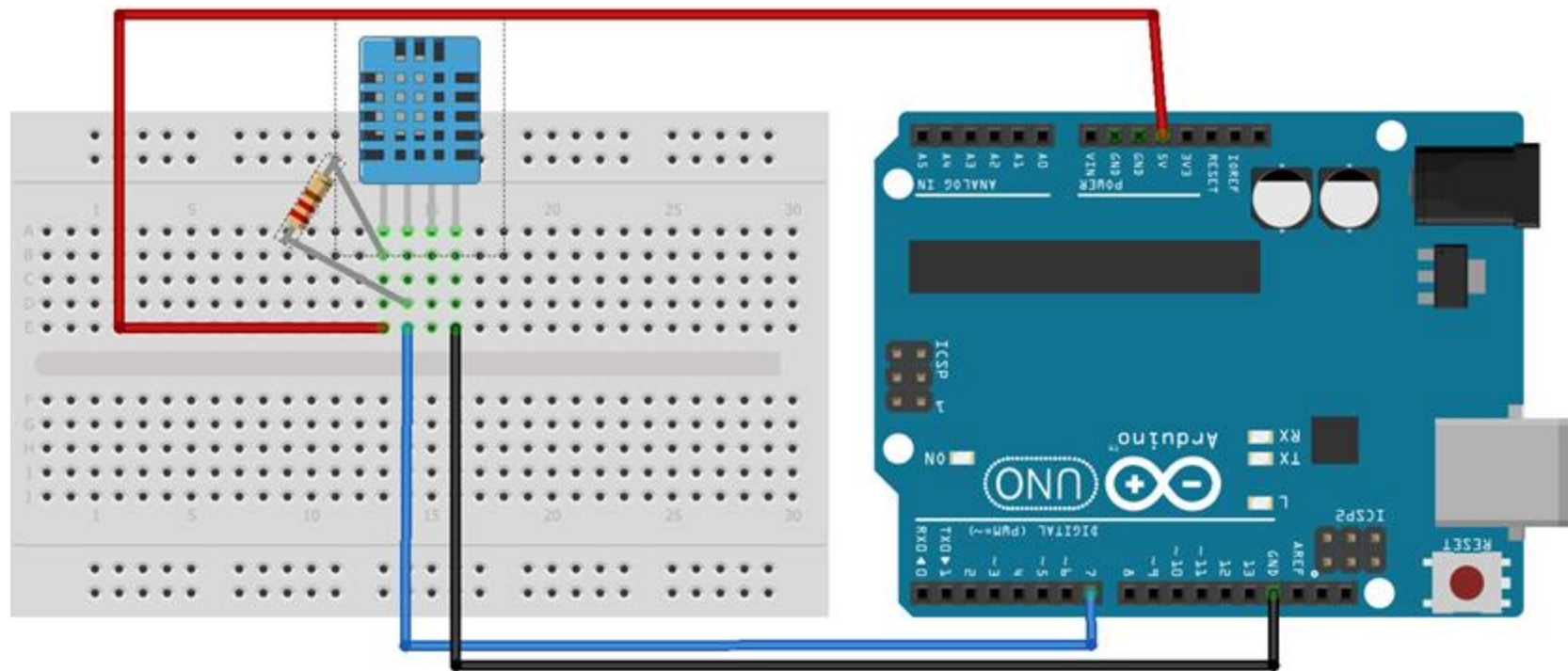


DHT11

- Float variable
 - Datatype for floating-point numbers, a number that has a decimal point. Floating-point numbers are often used to approximate analog and continuous values because they have greater resolution than integers.

```
DHT11.ino
6
7 created in Feb 2019 by D. Cuartielles
8 based on work by F. Vanzati (2011) and M. Loglio (2013)
9
10 This example code is in the public domain.
11 */
12
13 // include the EduIntro library
14 #include <EduIntro.h>
15
16 DHT11 dht11(D7); // creating the object sensor on pin 'D7'
17
18 int C; // temperature C readings are integers
19 float F; // temperature F readings are returned in float format
20 int H; // humidity readings are integers
21
22 void setup()
23 {
24     // initialize serial communications at 9600 bps
25     Serial.begin(9600);
26 }
27
28 void loop()
29 {
30     dht11.update();
31
32     C = dht11.readCelsius(); // Reading the temperature in Celsius degrees and store in the C variable
33     F = dht11.readFahrenheit(); // Reading the temperature in Fahrenheit degrees and store in the F variable
34     H = dht11.readHumidity(); // Reading the humidity index
35
36     // Print the collected data in a row on the Serial Monitor
37     Serial.print("H: ");
38     Serial.print(H);
39     Serial.print("\tC: ");
40     Serial.print(C);
41     Serial.print("\tF: ");
42     Serial.println(F);
43
44     delay(1000); // Wait one second before get another temperature reading
45 }
46
```

DHT11



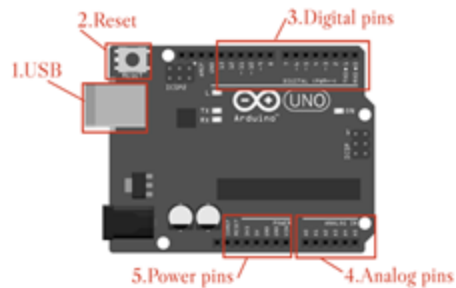
Putting it together



Arduino Cards



ARDUINO BOARD



1. Power and upload sketches to board
2. Starts sketch from the beginning
3. 13 digital pins, used both as inputs and outputs
4. 6 analog pins, work only as inputs
5. Power and ground

PIEZO

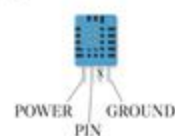


Speakers that can play various tones

EXAMPLES WHERE TO USE THEM

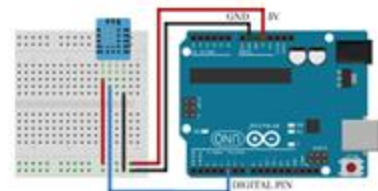
- Sound effects
- Making own music
- Confirmation of user input

EXAMPLE HOW TO WIRE IT



Every second DHT11 reads a new value:

	Range	Error
Temp	0-50°C	±2°C
Humid	20-80%	±5%



DHT11

DIGITAL HUMIDITY/TEMPERATURE SENSOR



Measure humidity
and temperature
levels

EXAMPLES WHERE TO USE THEM

- Automatic humidifier
- Weather station
- Controlling block heater for a car

?

Is it more than
+20 degrees inside
the room?

!

Turn on the fan

SERVO MOTOR



When you want to
move something
you need a motor

EXAMPLES WHERE TO USE THEM

- Robotic arm
- Open and close a door
- Barometer

Building complex programs

1. Think input - What will be the trigger, the interaction.
2. How am I going to Output?
3. The Brain between the senses is the Arduino board
 - a. Think Control Structures
 - i. If then and Else if...
 - b. Other ways see Switch Case (reference)
 - c. Remember LOOP goes on and on until the end
4. Use the Serial Monitor to check things...

Complexity

1. For some sensors using the **map() function** can be good
2. You may need to use **If then Else if** to build structure to your project
3. Consider how to **adjust your ideas** to convey the concept rather than being locked into a single way...

Structure Sketch

✓ → uno my uno ... Sketch

Declare =

```
#include <EduIntro.h>
→ inputs / outputs
DHT11 dht11(D7);
Servomotor servos1(D10);
Servo motor servos2(D11);
PINS ← D7, D10, D11

→ Variables (global)
int mything = 0;
int myArray[] { 4, 5, 8, 12, 53};
```

Setup

```
void setup() {
  Serial.begin(9600);
}
```

Loop

```
void loop() {
  if (condition == A) {
    do thing A
  }
  * elseif (condition == B) {
    do thing B
  }
  else (condition == C) {
    do thing C
  }
  * as many as you need
}
```



Your mission today:

Requirements for the toy

1. Target group should be kids, elderly people, chronically ill, or pets
2. At least one input (kit includes: buttons, light sensor, humidity/temperature sensor)
3. At least one output (kit includes LEDs, speaker, motor)



Your Resources:

1. Each other, TAs, Mentors, and me
2. Arduino and hardware
3. Arduino Learning Cards that will help you to program, wire and design the toy
4. Arduino Kit
5. Crafting materials to decorate your toy such as glue gun, paper, pens, etc.

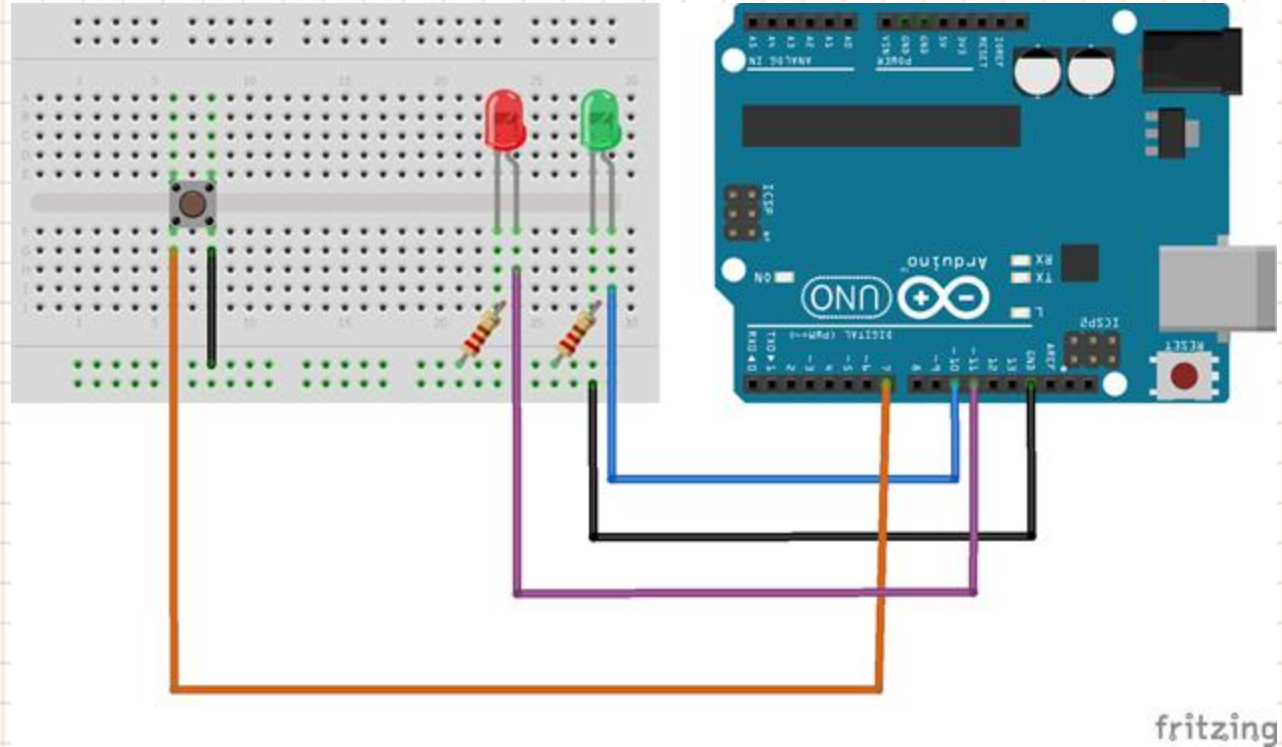


Brainstorm in teams of circa 3

1. Coder, Electronics, and Designer (your team roles)
2. Quickly 5 minutes brainstorm individually - as many ideas as possible
3. Now evaluate the ideas and choose 1 or 2 as a team
4. Sketch out your ideas with the cards
5. Make a plan
6. Build!

Extra goodies

More about Buttons & Serial Monitor



More about Buttons & Serial Monitor

The screenshot shows the Arduino IDE web editor interface. The left sidebar contains navigation links: EDITOR, Sketchbook, Examples, Libraries, Monitor, Help, and Preferences. The main editor area displays the sketch code for `Day1_ButtonStates_LED`. The code includes comments about button states and LED control. The bottom status bar shows a success message: "Success: Done uploading Day1_ButtonStates_LED".

```
Day1_ButtonStates_LED

1 // EduIntro - Version: 0.0.6
2 #include <EduIntro.h>
3 #include <pinches.h>
4
5 /*
6  ButtonStates
7
8  Test the different Button methods: pressed, released, held
9  and getSwitch.
10
11  This example code is in the public domain.
12
13  created in Aug 2018 by D. Cuartielles
14  based on work by F. Vanzati (2011) and M. Ioglio (2013)
15  */
16
17 #include <EduIntro.h>
18
19 Led led1(D10); // creating 1 object 'led' on pin D10
20 Led led2(D11); // creating 2 object 'led' on pin D11
21
22 Button btn(D7); // creating the object 'button' on pin D7
23
24 void setup()
25 {
26   // we are going to use the serial communication as a
27   // way to see on the PC what is happening on the Arduino
28   Serial.begin(9600);
```

Success: Done uploading Day1_ButtonStates_LED

avridude: safemode efuse reads as 0
avridude: safemode Fuses OK (E 00, H 00, L 00)
avridude done. Thank you.

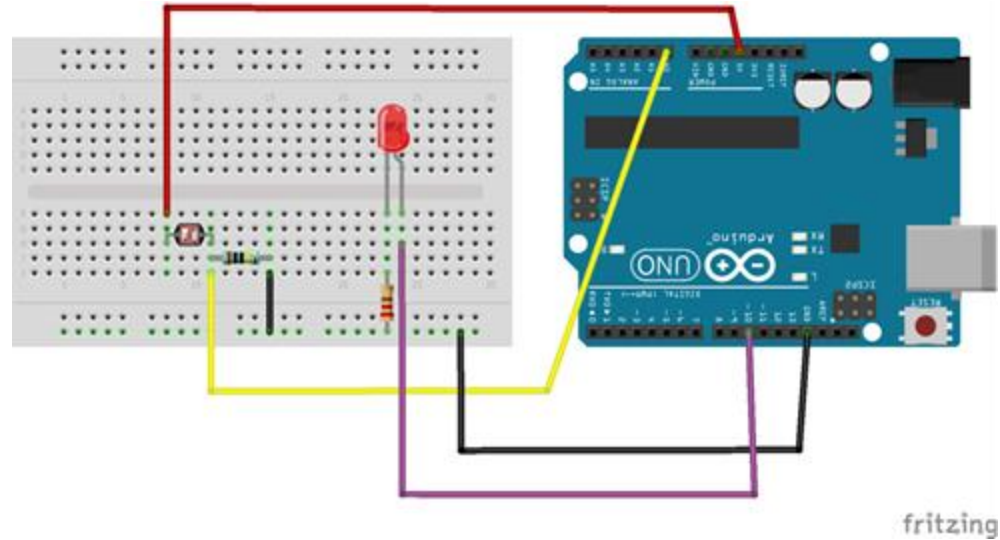
Light Dependent Resistors



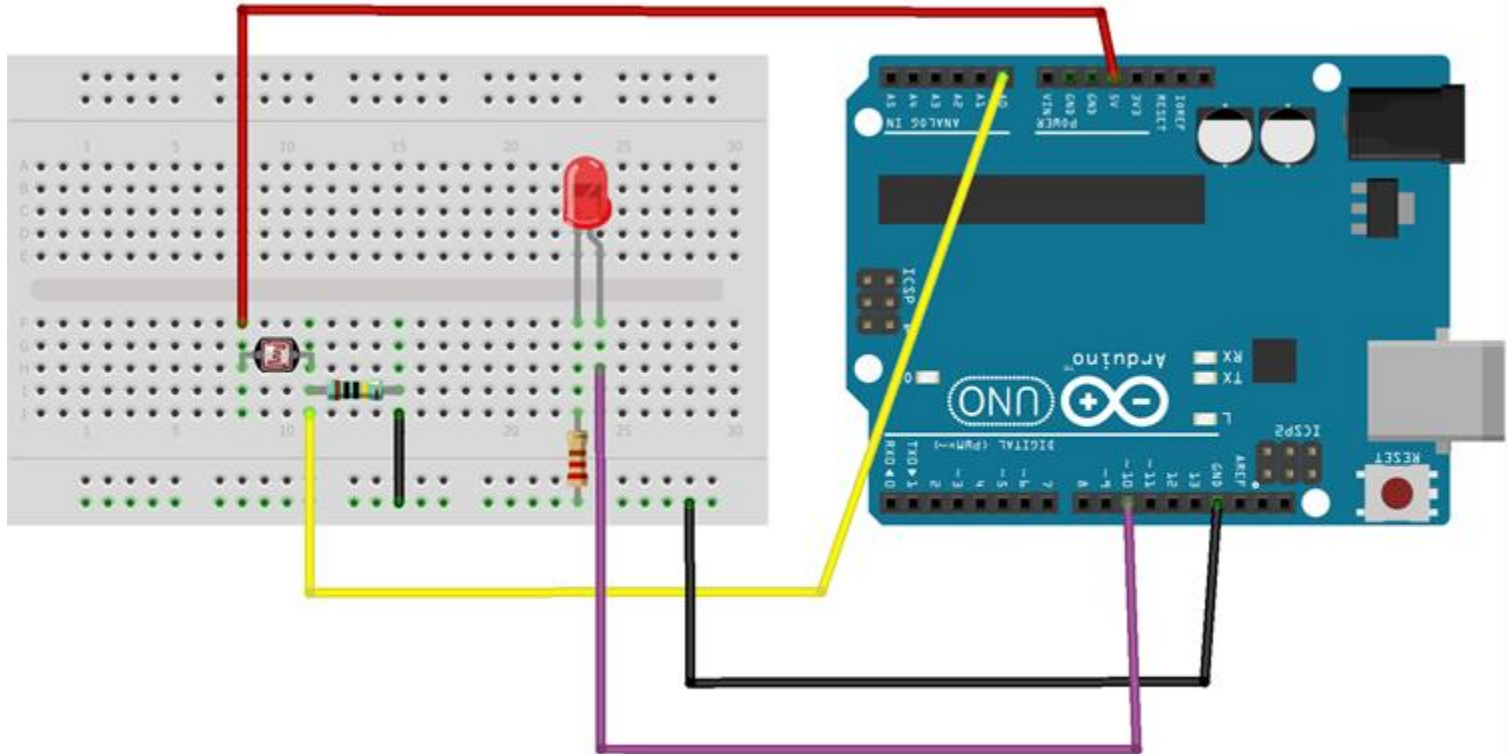
Use the LDR to control the LED

- Ingredients
 - LED
 - 200 Ohm resistor
 - LDR (Big)
 - 1M Ohm resistor (blue)
- Cooking

EDUINTRO >> by_topic >>
LightSensor



Use the LDR to control the LED



Concepts for the LDR to control the LED

- Concepts

- experiment with `In 33`
`//led.brightness(1023 - brightnessVal);` see what happens when you uncomment that.
- notice that the LDR sensor is analogue and requires 5 volts
- see that we use the `setup()` to start the Serial.Monitor

- Concepts

- Go a step further by adding a `if.. then` statement with the variable `brightnessVal`
- See an example of things at <https://create.arduino.cc/editor/danielspikol/fb3f8366-c4f9-4559-8331-bd07eb9e28eb/preview>

```
LightSensor.ino
14
15 LightSensor ldr(A0); //create the "ldr" object on pin A0
16
17 Led led(D10); //create the "led" object on pin D10
18
19 void setup() {
20   // initialize serial communications at 9600 bps
21   Serial.begin(9600);
22 }
23
24 void loop() {
25   // store the ldr values into a variable called brightnessVal
26   int brightnessVal = ldr.read();
27
28   // set the led brightness
29   led.brightness(brightnessVal);
30
31   //to have it at full brightness
32   //when it's dark, uncomment this line:
33   //led.brightness(1023 - brightnessVal);
34
35   // print the results to the serial monitor:
36   Serial.print("brightness = ");
37   Serial.println(brightnessVal);
38
39
40   // wait 10 milliseconds before the next loop
41   delay(10);
42 }
43
```

Programming

int Variable

Description

Integers are your primary data-type for number storage.

Syntax

```
int var = val;
```

var - your int variable name

val - the value you assign to that variable

See <https://www.arduino.cc/reference/en/language/variables/data-types/int/>

180 Potentiometer

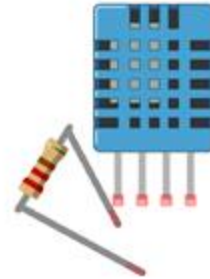
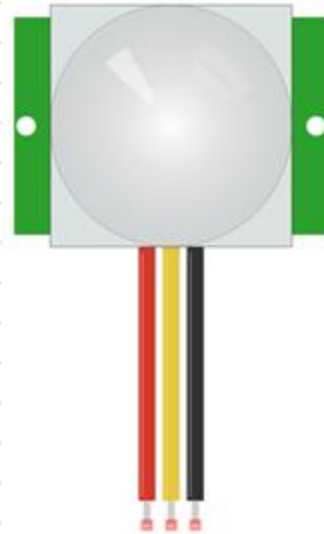
- Concepts
 - Need to declare the Potentiometer (Ln 14)
 - Need to read the value (Ln28)
- Map() function
 - Re-maps a number from one range to another. That is, a value of **fromLow** would get mapped to **toLow**, a value of **fromHigh** to **toHigh**, values in-between to values in-between, etc.
 - <https://www.arduino.cc/reference/en/language/functions/math/map/>
 - Use of local variable "angleVal" (Ln 32)

```
ServoKnob.ino
14 Potentiometer pot(A0); // creating the object 'pot' on pin A0
15 ServoMotor servo(D10); // creating the object 'servo' on pin D10
16
17 int potVal = 0; // value read from the pot
18 int angleVal = 0; // value of the angle to set the servo
19
20 void setup() {
21   // initialize serial communications at 9600 bps
22   Serial.begin(9600);
23 }
24
25 void loop()
26 {
27   // read the potentiometer's value:
28   potVal = pot.read();
29
30   // map the data from the sensor (0..1023) to
31   // one the servo can take (0..180)
32   angleVal = map(potVal, 0, 1023, 0, 180);
33
34   // set the led brightness
35   servo.write(angleVal);
36
37   // print the results to the serial monitor:
38   Serial.print("angle = ");
39   Serial.println(angleVal);
40   // wait 100 milliseconds before the next loop
41   delay(100);
42 }
43
```

Pryoelectric Infrared Sensors



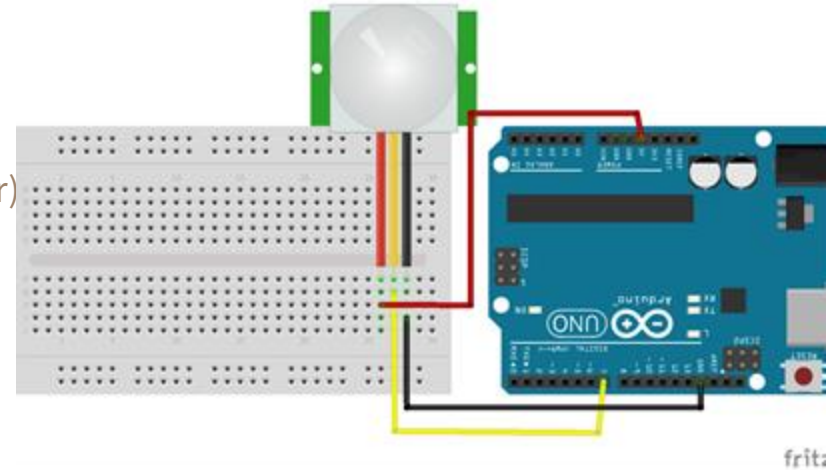
Today's components



fritzing

P.I.R.

- Cooking
 - PIR sensor
 - 5V, Signal, Gnd
 - LED required (D10 and 220 Resistor)
- Ingredients
 - EDUINTRO>>by_topic>>PIR
 - EDUINTRO>>by_topic>>PIRStates



P.I.R.

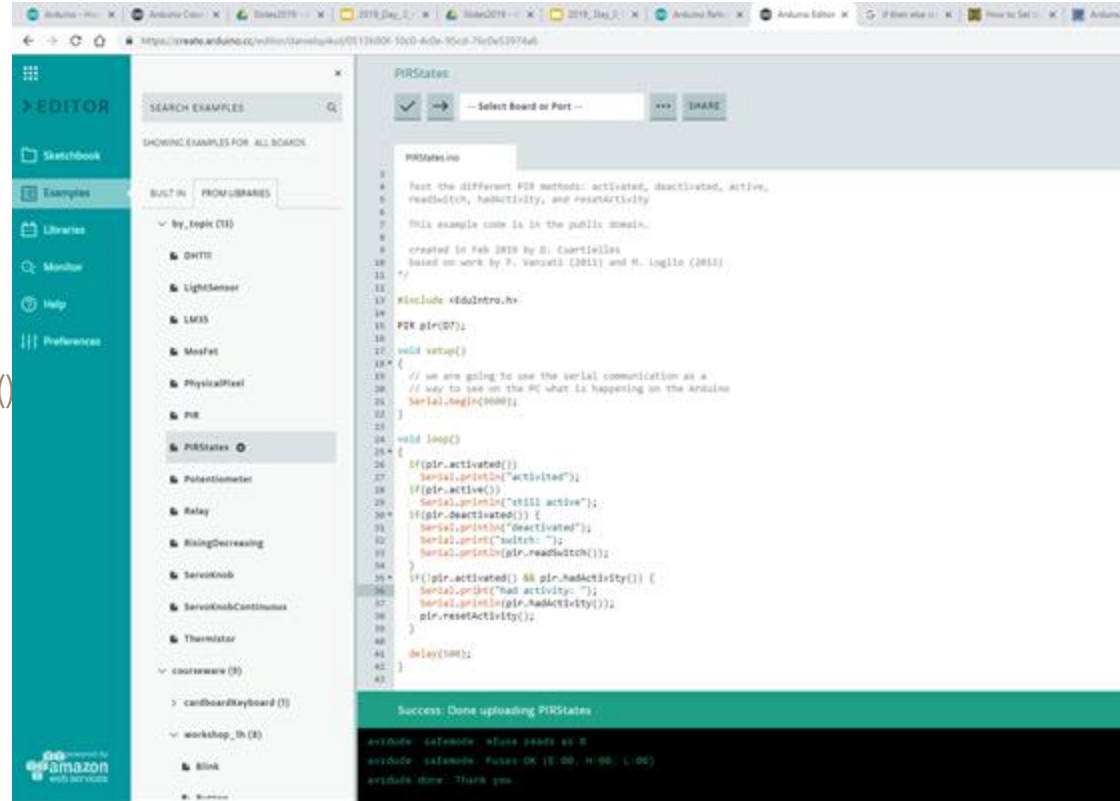
- Concepts
 - pir.active()
 - pir.activated()

```
PIR.ino

1 /*
2  PIR
3
4  Changes the behavior between on and off an LED when a PIR sensor is activated.
5  Assumes the use of a PIR sensor that needs no pull-up
6
7  This example code is in the public domain.
8
9  created in Feb 2019 by D. Cuartielles
10 based on work by F. Vanzati (2011) and M. Loglio (2013)
11 */
12
13 // include the EduIntro library
14 #include <EduIntro.h>
15
16 PIR pir(D7); // creating the object 'pir' on pin D7
17
18 Led led(D10); // creating the object 'led' on pin D10
19
20
21 void setup() {
22   //nothing here
23 }
24
25 void loop()
26 {
27   // check the state of the PIR sensor
28   // if you just want to detect the turn from not active to active
29   // use pir.activated() instead
30   if (pir.active() == HIGH) {
31     led.on();
32   }
33   else {
34     led.off();
35   }
36 }
37
```

PIRSTATES

- Concepts
 - `pir.active()`
 - `pir.activated()`
 - `pir.deactivated()`

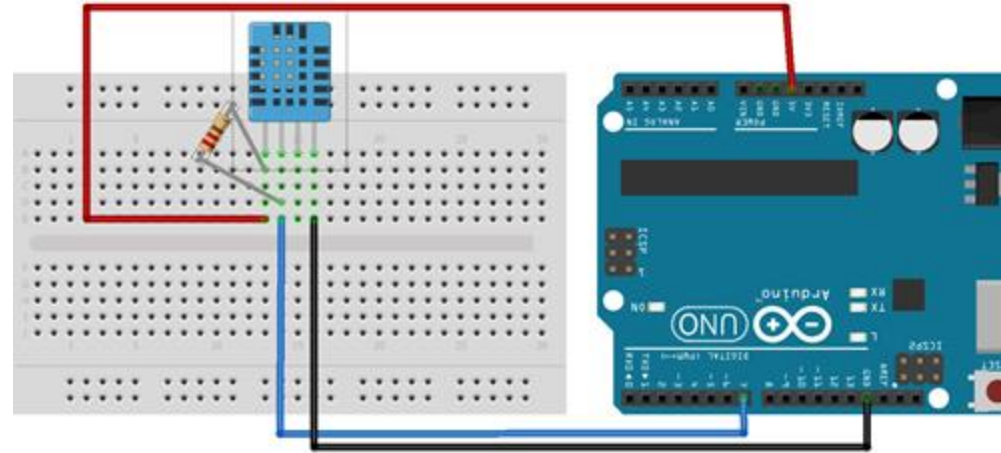


Digital Humidity and Temperature DHT11 Sensor



DHT11

- Cooking
 - DHT11 Sensor
 - Resistor (in bag)
 - Between 5V and Signal
- Ingredients
 - EDUINTRO>>by_topic>>DHT11



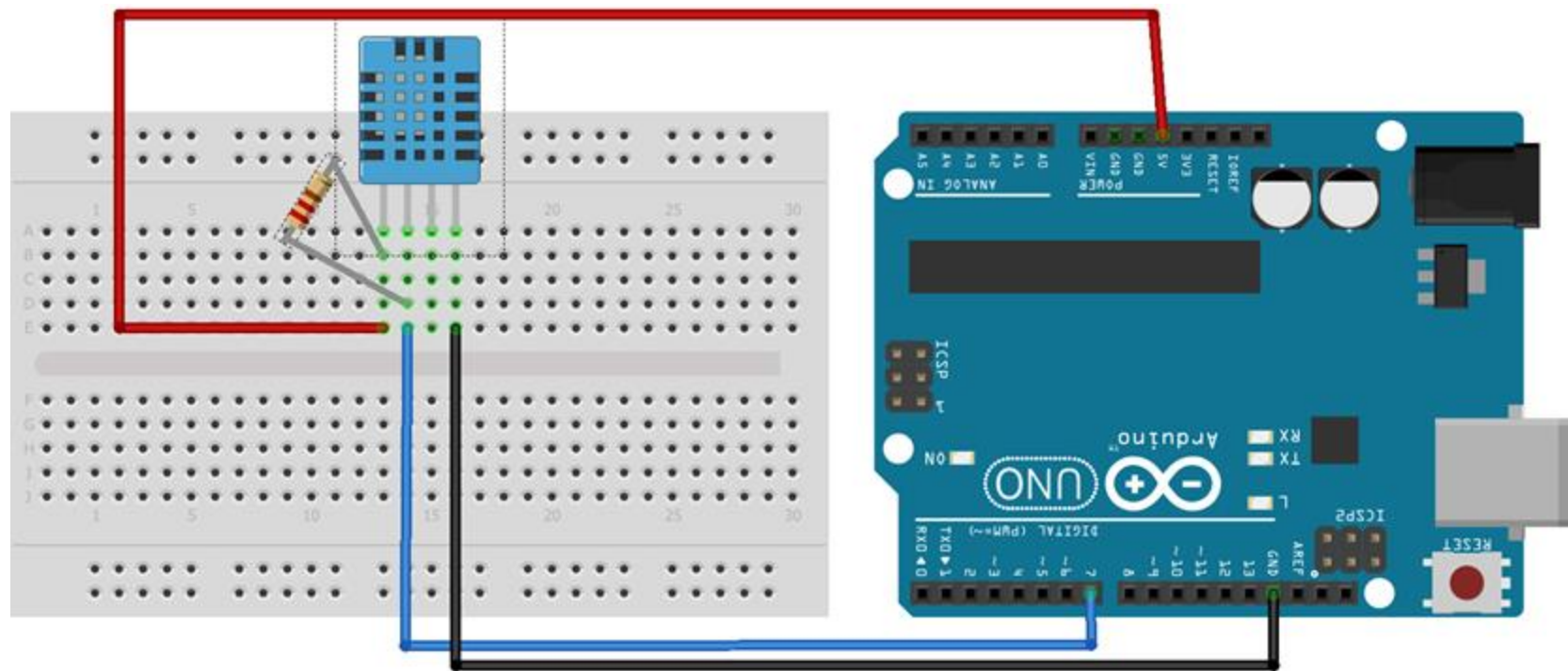
DHT11

- Concepts

- Float
 - Datatype for floating-point numbers, a number that has a decimal point. Floating-point numbers are often used to approximate analog and continuous values because they have greater resolution than integers.

```
DHT11ino
6
7 created in Feb 2019 by D. Cuartielles
8 based on work by F. Vanzati (2011) and M. Loglio (2013)
9
10 This example code is in the public domain.
11 */
12
13 // include the EduIntro library
14 #include <EduIntro.h>
15
16 DHT11 dht11(D7); // creating the object sensor on pin 'D7'
17
18 int C; // temperature C readings are integers
19 float F; // temperature F readings are returned in float format
20 int H; // humidity readings are integers
21
22 void setup()
23 {
24     // initialize serial communications at 9600 bps
25     Serial.begin(9600);
26 }
27
28 void loop()
29 {
30     dht11.update();
31
32     C = dht11.readCelsius(); // Reading the temperature in Celsius degrees and store in the C variable
33     F = dht11.readFahrenheit(); // Reading the temperature in Fahrenheit degrees and store in the F variable
34     H = dht11.readHumidity(); // Reading the humidity index
35
36     // Print the collected data in a row on the Serial Monitor
37     Serial.print("H: ");
38     Serial.print(H);
39     Serial.print("\tC: ");
40     Serial.print(C);
41     Serial.print("\tF: ");
42     Serial.println(F);
43
44     delay(1000); // Wait one second before get another temperature reading
45 }
46
```

DHT11



IDE Download version



Download the IDE

- Download IDE from:
<https://www.arduino.cc>
- First let's blink
- Library needed also (next slides)



Download the IDE

- Download IDE from:
<https://www.arduino.cc>
- First let's blink
- Library needed also (next slides)



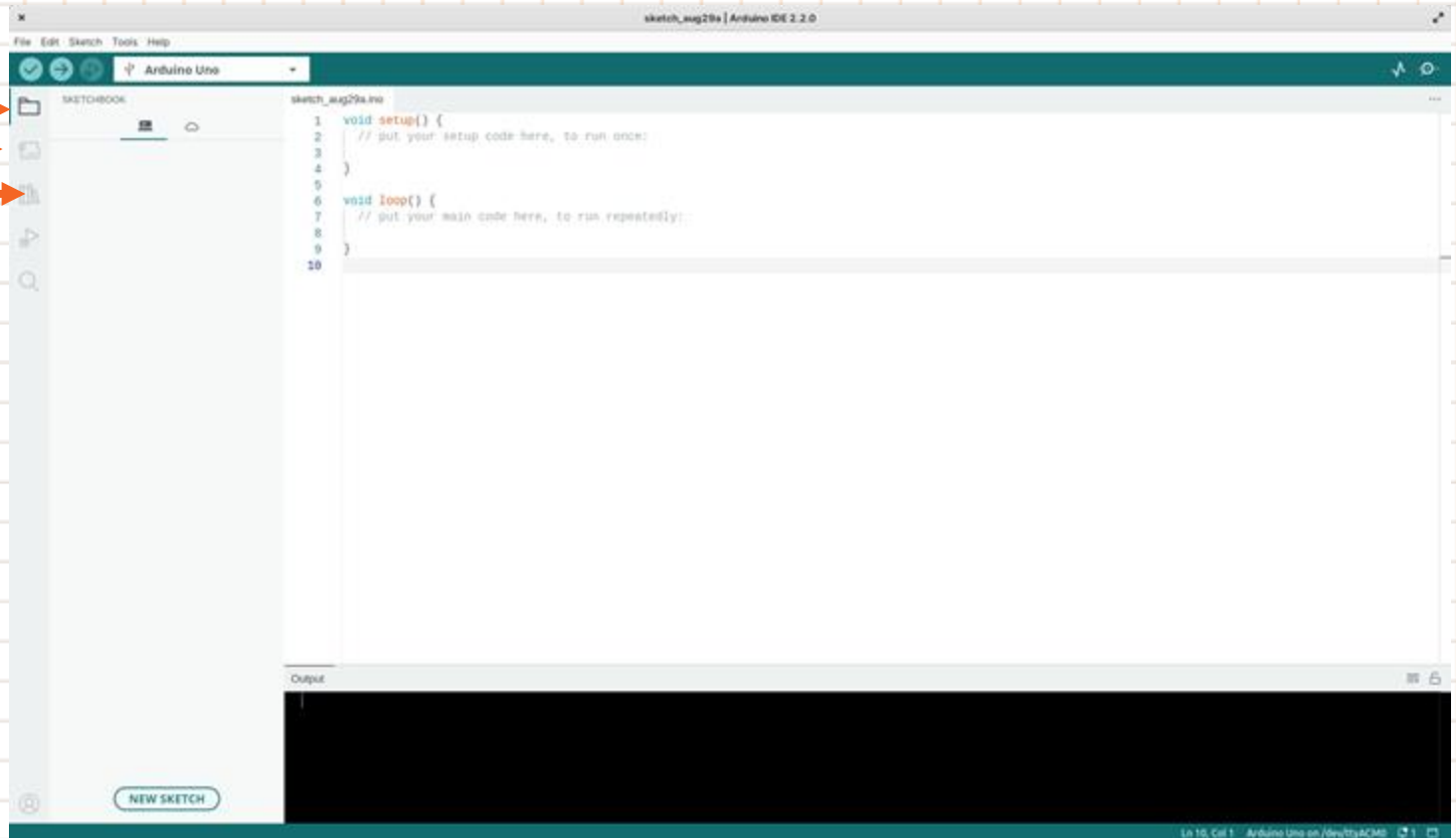
Arduino IDE

Sketches

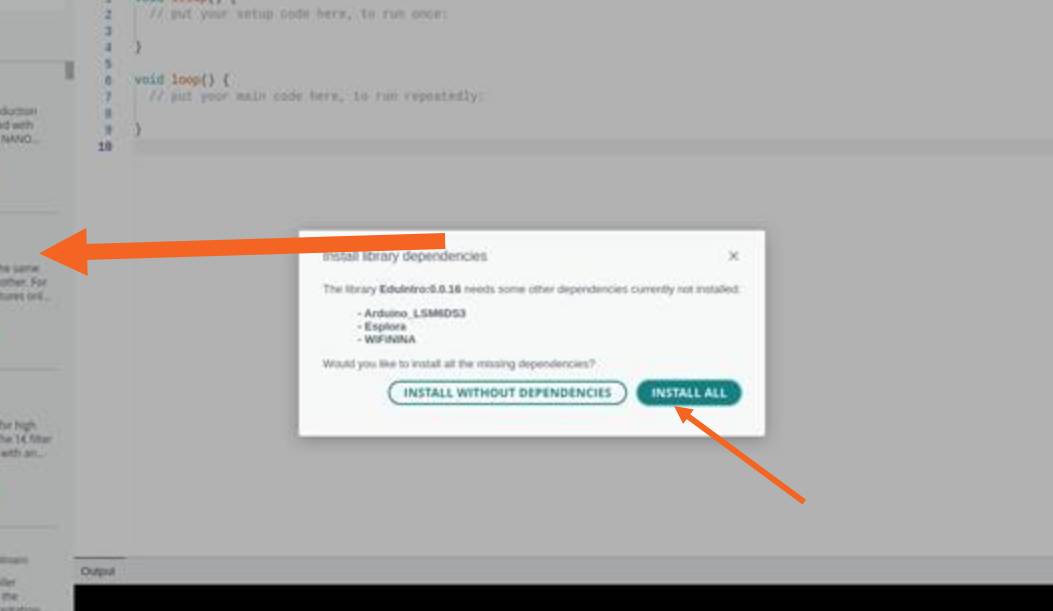
Boards

Libraries

Debugging



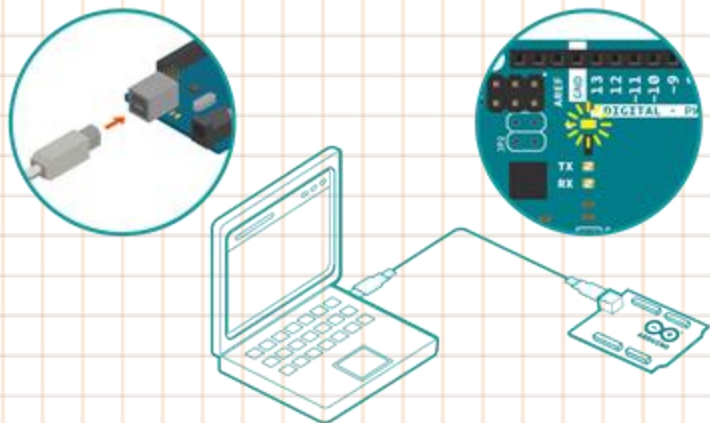
Install EduIntro Library



The screenshot shows the Arduino IDE interface. In the Library Manager, the 'EduIntro' library by Arduino LLC is selected. The library description states it is for super-fast introduction workshops and is intended to be used with Arduino UNO / MICRO / MEGA / NANO. The version 0.0.18 is shown, and the 'INSTALL' button is highlighted with a red arrow. A dialog box titled 'INSTALL library dependencies' is open, showing the dependencies: Arduino_LSMDS3, Esp8266, and WiFiNINA. The dialog asks 'Would you like to install all the missing dependencies?' and has two buttons: 'INSTALL WITHOUT DEPENDENCIES' and 'INSTALL ALL'. A red arrow points to the 'INSTALL ALL' button.

Arduino IDE Library Manager interface showing the 'EduIntro' library by Arduino LLC. The library is selected, and the 'INSTALL' button is highlighted. A red arrow points to the 'INSTALL' button. A dialog box titled 'INSTALL library dependencies' is open, showing the dependencies: Arduino_LSMDS3, Esp8266, and WiFiNINA. The dialog asks 'Would you like to install all the missing dependencies?' and has two buttons: 'INSTALL WITHOUT DEPENDENCIES' and 'INSTALL ALL'. A red arrow points to the 'INSTALL ALL' button.

Arduino Blink



```
Arduino IDE 2.2.0
File Edit Sketch Tools Help
Arduino Uno

1 FILE>
  EXAMPLES>
    BASICS>
      BLINK
        Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO
        it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to
        the correct LED pin independent of which board is used.
        If you want to know what pin the on-board LED is connected to on your Arduino
        model, check the Technical Specs of your board at:
        https://www.arduino.cc/en/Main/Products
        modified 8 May 2014
        by Scott Fitzgerald
        modified 2 Sep 2016
        by Arturo Guadalupi
        modified 8 Sep 2016
        by Polby Newman

2 TOOLS>
  BOARD>
    ARDUINO UNO

3 TOOLS>
  SERIAL PORT
    COM####
    MacOS /dev/cu.usbmodem#### the public domain.
    Linux OS /dev/ttyACM#### https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink

4 FILE>
  UPLOAD
    function runs once when you press reset or power the board
    initialize digital pin LED_BUILTIN as an output.
    pinMode(LED_BUILTIN, OUTPUT);

    // the loop function runs over and over again forever
    void loop() {
      digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
      delay(1000); // wait for a second
      digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW
      delay(1000); // wait for a second
    }
  }
```

The IDE MENU

File Menu

New, Save, Open,
Examples...

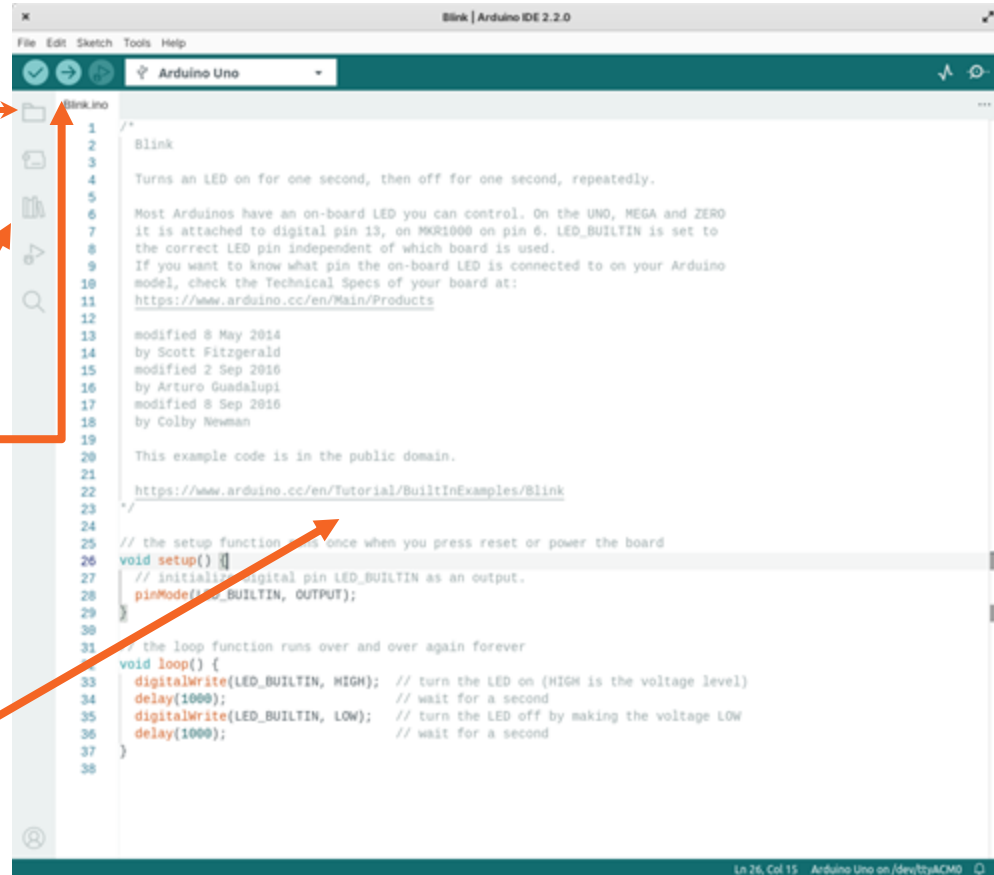
Verify and Upload

Test your code
Upload you code +

COM-Port Settings,
Tools, Board Type...

Code Area

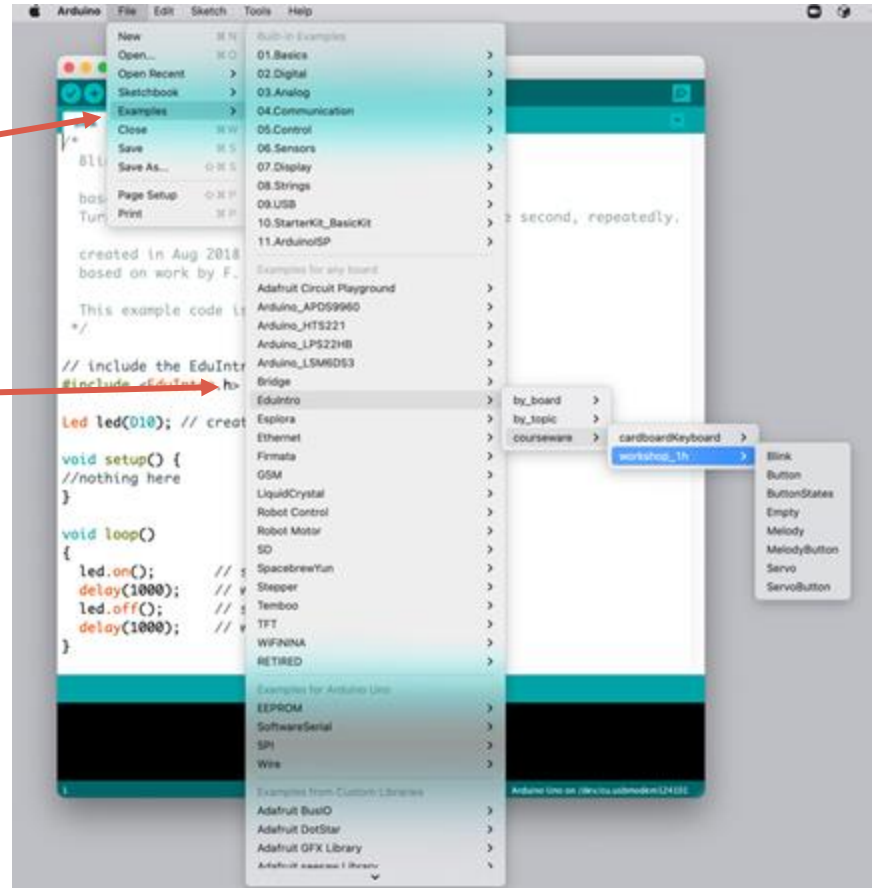
Cut, Copy, Paste



The IDE Functions

Sketches and Tools

Library & Examples



Programming and Boards

